

Gustavo Moreira Calixto

Práticas de **programação**

Dados Internacionais de Catalogação na Publicação (CIP)
(Claudia Santos Costa - CRB 8ª/9050)

Calixto, Gustavo Moreira

Práticas de programação / Gustavo Moreira Calixto. – São Paulo :
Editora Senac São Paulo, 2025. – (Série Universitária)

Bibliografia.

e-ISBN 978-85-396-5539-7 (ePub/2025)

e-ISBN 978-85-396-5540-3 (PDF/2025)

1. Programação (Computadores). 2. Python (Linguagem de
programação). I. Título. II. Série..

25-2460c

CDD – 005.13

BISAC COM051000

COM051360

Índice para catálogo sistemático:

1. Programação (Computadores) 005.13

PRÁTICAS DE PROGRAMAÇÃO

Gustavo Moreira Calixto





Administração Regional do Senac no Estado de São Paulo

Presidente do Conselho Regional

Abram Szajman

Diretor do Departamento Regional

Luiz Francisco de A. Salgado

Superintendente Universitário e de Desenvolvimento

Luiz Carlos Dourado

Editora Senac São Paulo

Conselho Editorial

Luiz Francisco de A. Salgado

Luiz Carlos Dourado

Darcio Sayad Maia

Lucila Mara Sbrana Sciotti

Luís Américo Tousi Botelho

Gerente/Publisher

Luís Américo Tousi Botelho

Coordenação Editorial

Verônica Marques Pirani

Prospecção

Andreza Fernandes dos Passos de Paula

Dolores Crisci Manzano

Paloma Marques Santos

Administrativo

Marina P. Alves

Comercial

Aldair Novais Pereira

Comunicação e Eventos

Tania Mayumi Doyama Natal

Coordenação de Arte

Antonio Carlos De Angelis

Coordenação de Revisão de Texto

Marcelo Nardeli

Acompanhamento Pedagógico

Otacília da Paz Pereira

Designer Educacional

Janaina Nascimento Menezes da Silva

Revisão Técnica

Alexandre dos Santos Mignon

Preparação e Revisão de Texto

Allan Moraes

Amanda Carvalho

Projeto Gráfico

Alexandre Lemes da Silva

Emília Corrêa Abreu

Capa

Antonio Carlos De Angelis

Editoração Eletrônica e Ilustrações

Natália da Silva Nakashima

Manuela Ribeiro

Imagens

Adobe Stock Photos

Proibida a reprodução sem autorização expressa.

Todos os direitos desta edição reservados à

Editora Senac São Paulo

Av. Engenheiro Eusébio Stevaux, 823 – Prédio Editora

Jurubatuba – CEP 04696-000 – São Paulo – SP

Tel. (11) 2187 4450

editora@sp.senac.br

<https://www.editorasenacsp.com.br>

© Editora Senac São Paulo, 2025

Sumário

Capítulo 1 Programação estruturada, 7

- 1 Funções e procedimentos, 8
- 2 Escopo de variáveis, 13
- Considerações finais, 15
- Referências, 16

Capítulo 2 Listas, 17

- 1 Estruturas unidimensionais, 19
- 2 Operações básicas, 22
- 3 Listas e funções, 25
- 4 Estruturas bidimensionais, 27
- Considerações finais, 29
- Referências, 30

Capítulo 3 Fatiamento (slicing), 31

- 1 Fatiamento de listas, 32
- 2 Operações com strings, 35
- Considerações finais, 38
- Referências, 38

Capítulo 4 Tuplas e dicionários, 39

- 1 Tuplas, 40
- 2 Dicionários, 43
- 3 Tuplas e dicionários em funções, 48
- Considerações finais, 51
- Referências, 52

Capítulo 5 Função lambda, 53

- 1 Conceito e aplicações, 54
- 2 Uso das funções sort, map e list, 56
- Considerações finais, 61
- Referências, 61

Capítulo 6 Tratamento de exceções, 63

- 1 Conceito e aplicações, 64
- 2 Invocando exceções manualmente, 67
- Considerações finais, 70
- Referências, 70

Capítulo 7 Arquivos, 71

- 1 Arquivos texto, 72
- 2 Arquivos binário, 78
- Considerações finais, 80
- Referências, 81

Capítulo 8 Gerenciamento de módulos, 83

- 1 A comunidade de desenvolvimento Python, 84
- 2 O gerenciamento de módulos com pip, 86
- 3 Ambiente virtual, 88
- 4 Criando seus próprios módulos, 89
- Considerações finais, 94
- Referências, 95

Sobre o autor, 97

Programação estruturada

O aprendizado sobre programação é essencial para os profissionais da área de TIC (tecnologias da informação e comunicação) aperfeiçoarem suas habilidades no uso de ferramentas de elaboração, aplicações e serviços advindas do avanço da computação nos tempos atuais. Programar, hoje em dia, também é uma competência trabalhada em jovens do ensino básico, o que revela que a interação com elementos da computação não está limitada a profissionais de TIC.

Alguns fundamentos de programação são importantes para alcançar um nível básico de conhecimento na área, como a definição de variáveis, entrada e saída de dados, desvios condicionais e laços. No entanto, à medida que os programas se tornam mais complexos, é necessário adotar técnicas que permitam otimizar e reutilizar os artefatos desenvolvidos. De acordo com Forbellone e Eberspächer (2022), o uso de estruturas modulares e a organização do código são fundamentais para garantir a eficiência e a manutenção dos programas, facilitando sua evolução e adaptação a novas demandas.

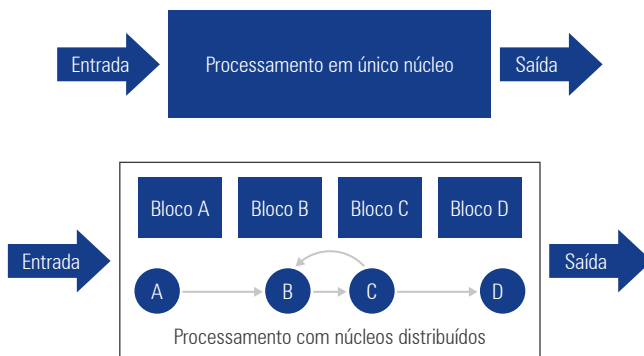
Sem dúvida, a primeira prática para estudantes que desejam alcançar o nível intermediário em programação é a organização de seu código-fonte através de funções e procedimentos, abordagem conhecida como programação estruturada. O primeiro subcapítulo deste livro conceitua e apresenta exemplos de funções e procedimentos, enquanto o segundo mostra os diferentes escopos de variáveis em programas que trabalham com um ou mais subprogramas.

1 Funções e procedimentos

Os algoritmos são conjuntos de instruções finitas organizadas para um propósito específico, como aperfeiçoar tarefas, processar grandes volumes de dados ou monitorar sensores e atuadores em qualquer segmento para uma aplicação ou serviço.

Existem programas desenvolvidos em poucas linhas cuja sequência de instruções é claramente interpretável. Contudo, podem existir situações em que o código-fonte se torna extenso, incorporado a um núcleo único de processamento, com a repetição das mesmas instruções para realizar tarefas idênticas ou similares. Quando um programa chega a esse nível de complexidade, é necessário buscar estratégias para atribuir as devidas responsabilidades a cada trecho de código, evidenciando as etapas do processamento elaborado. A figura 1 ilustra a condição de um programa cujas instruções estão agrupadas em um único núcleo e a organização que se pretende alcançar. Observe que um mesmo programa pode ter sua responsabilidade descentralizada em blocos que podem ser invocados na ordem e quantidade de vezes estabelecidas pelo desenvolvedor. Práticas de programação estruturada favorecem a organização do código e facilitam o entendimento para futuras ações de refatoração (melhora do código-fonte sem alteração de suas funcionalidades).

Figura 1 – Comparação entre programa com núcleo único e programa com núcleos distribuídos



Um exemplo prático é apresentado para um programa que verifica as aprovações de alunos informando o nome e a nota.

```
# Código sem modularização (bloco único)
nome = "João"
nota1 = 8.0
nota2 = 7.5
nota3 = 6.0

media = (nota1 + nota2 + nota3) / 3

if media >= 7.0:
    resultado = "Aprovado"
else:
    resultado = "Reprovado"

print(f"Aluno: {nome}")
print(f"Média: {media:.2f}")
print(f"Resultado: {resultado}")
```

Observe que esse programa utiliza práticas básicas de entrada, variáveis, desvio condicional e saída em bloco único. O código-fonte pode crescer para processar em maior volume e detalhes de dados, exigindo-se para isso a distribuição de responsabilidades. Veja, portanto, o mesmo

código com uma abordagem de programação estruturada reorganizado em funções e procedimentos:

```
# Código otimizado com funções

def calcular_media(n1, n2, n3):
    """Calcula a média de três notas."""
    return (n1 + n2 + n3) / 3

def verificar_aprovacao(media, media_aprovacao=7.0):
    """Verifica se a média é suficiente para aprovação."""
    return "Aprovado" if media >= media_aprovacao else "Reprovado"

def exibir_resultado(nome, media, resultado):
    """Exibe o nome do aluno, sua média e o status de aprovação."""
    print(f"Aluno: {nome}")
    print(f"Média: {media:.2f}")
    print(f"Resultado: {resultado}")

# Variáveis
nome = "João"
nota1 = 8.0
nota2 = 7.5
nota3 = 6.0

# Processamento
media = calcular_media(nota1, nota2, nota3)
resultado = verificar_aprovacao(media)
exibir_resultado(nome, media, resultado)
```

Constate que a resolução do programa é dividida em três partes: `calcular_media`, `verificar_aprovacao` e `exibir_resultados`. Logo após, essas partes são invocadas de maneira a efetuar o processamento desejado. Para as práticas de programação em programas complexos, a divisão de responsabilidades em funções e procedimentos resulta em uma melhor organização e, na maioria dos casos, contribui para um desempenho mais eficiente. Segundo Guilhon *et al.* (2022), a modularização do código

é uma abordagem essencial para garantir a escalabilidade e a manutenibilidade das aplicações, permitindo que diferentes partes do programa sejam desenvolvidas, testadas e aprimoradas de forma independente.

Perceba que, no código-fonte do exemplo apresentado, a função “verificar_aprovacao” possui um parâmetro de entrada “media_aprovacao = 7.0”. Essa prática é chamada de parâmetros default ou parâmetros padrão e permite adotar o valor informado como padrão para executar a função, caso não seja informado um valor para esse parâmetro. No caso da função “verificar_aprovacao”, observe que ela é executada sem que se informe o segundo parâmetro “media_aprovacao”, adotando o valor 7.0 como padrão. No entanto, se a média de aprovação fosse diferente de 7.0, o novo valor poderia ser informado no segundo parâmetro.

Conceitualmente, qual é a diferença entre uma função e um procedimento? Ambos são subprogramas na programação estruturada, mas possuem diferenças. Uma função é um bloco de código reutilizável que executa uma tarefa específica. As funções ajudam a organizar o código, tornando-o mais modular, reutilizável e legível. A sintaxe básica para definir uma função em Python é:

```
def nome_da_funcao(parametro1, parametro2, ...):  
    """Docstring opcional explicando a função"""  
    # Bloco de código da função  
    return resultado
```

É importante ressaltar que uma função (como nos ensinamentos da matemática) necessita de um retorno: um resultado do processamento realizado no subprograma. A função também pode receber parâmetros para o fornecimento de elementos a serem utilizados na execução do subprograma. Veja o exemplo a seguir do uso de uma função:

```
# Definição da função  
def saudacao(nome):  
    """Retorna uma saudação personalizada para o usuário."""  
    return f"Olá, {nome}! Bem-vindo ao mundo da programação."
```

```
# Chamando a função e exibindo o resultado
usuario = "Alice"
mensagem = saudacao(usuario)
print(mensagem)
```

Observe que a função retorna uma mensagem de saudação que possui como parâmetro o nome, ou seja, ela depende do nome para a devida execução. Na programação estruturada, é natural que, primeiramente, as funções e os procedimentos sejam projetados e desenvolvidos para depois serem invocados nos pontos desejados do programa principal.

Já o procedimento é uma função que executa uma ação, mas não retorna um valor explícito. A sintaxe de um procedimento é a mesma de uma função, mas sem a expressão *return*.

```
def nome_do_procedimento(parametros):
    '''Executa uma ação sem retornar um valor'''
    # Bloco de código da função
```

Os procedimentos são uma maneira de construir subprogramas quando o propósito é o retorno direto para a saída do programa, como apresenta o exemplo a seguir. Observe que a saudação é exibida diretamente por um comando de saída de terminal sem o uso da palavra reservada *return*.

```
# Definição do procedimento
def exibir_saudacao(nome):
    """Exibe uma saudação na tela sem retornar um valor."""
    print(f"Olá, {nome}! Seja bem-vindo!")

# Chamando o procedimento
exibir_saudacao("Lucas")
exibir_saudacao("Mariana")
```



IMPORTANTE

O uso de funções e procedimentos em Python é fundamental para a organização, a reutilização e a manutenção do código, pois permite dividir um programa em blocos menores e mais compreensíveis. Funções são a opção ideal quando há necessidade de calcular e retornar valores, enquanto procedimentos executam ações sem necessariamente retornar algo. Essa separação melhora a legibilidade, evita repetições desnecessárias e facilita a depuração, tornando o código mais modular e eficiente. Em aplicações grandes, essa abordagem reduz a complexidade, favorece o reaproveitamento de código e permite que diferentes partes do sistema sejam desenvolvidas e testadas de forma independente. É importante reforçar que, conceitualmente, na linguagem de programação Python, os procedimentos são adaptações em uma única estrutura de função. Assim, mesmo que o código-fonte do procedimento não tenha a expressão *return*, o valor "None" é retornado ao final da execução.

2 Escopo de variáveis

Ao trabalhar com funções e procedimentos, é importante entender que os recursos computacionais alocados por subprogramas são liberados ao final da sua execução dentro de um contexto de seu processo. Em linhas gerais, uma função ou procedimento pode ter variáveis declaradas dentro do seu bloco de execução, limitadas ao uso dentro do seu bloco. Todavia, esse subprograma pode ter acesso a variáveis declaradas no bloco principal de execução do programa. Observe o exemplo a seguir:

```
# Definição de uma variável global
mensagem_global = "Olá, bem-vindo ao programa!"

def saudacao(nome):
    """Função que usa uma variável global e uma variável
    local."""
```

```

    mensagem_local = f"Olá, {nome}! Tenha um ótimo dia!" #
Variável local
    print(mensagem_global) # Acessando a variável global
    print(mensagem_local) # Acessando a variável local

# Chamando a função
saudacao("Lucas")

```

As variáveis podem ser classificadas em dois tipos: globais e locais. As variáveis globais são declaradas fora das funções e podem ser acessadas de qualquer parte do código, inclusive dentro de funções e procedimentos. Por outro lado, as variáveis locais são definidas dentro de uma função e só podem ser utilizadas no seu escopo. Segundo Lambert (2022), compreender essa distinção é essencial para evitar conflitos no código e garantir que as variáveis sejam manipuladas de forma segura e eficiente.

Veja que, no exemplo apresentado, a variável `mensagem_global` pode ser acessada dentro da função `saudacao`. Já a variável `mensagem_local` só pode ser utilizada durante a execução do bloco da função. Se ao final do código do exemplo anterior fosse adicionado o código a seguir, um erro seria gerado por tentativa de acesso a uma variável local no escopo de uma variável global.

```

# Tentando acessar a variável local fora da função (isso
# gerará um erro)

# print(mensagem_local) # Isso resultaria em um erro pois a
# variável é local

```



IMPORTANTE

O uso de variáveis globais em Python pode parecer conveniente, mas apresenta diversos riscos que comprometem a qualidade do código. Como podem ser acessadas e modificadas de qualquer parte do programa, elas dificultam a depuração e, consequentemente, a identificação da origem de erros. Além disso, criam dependências ocultas, tornando as funções menos previsíveis e reduzindo sua reutilização. Outro problema é o risco

de sobrescrita acidental, o que pode levar a comportamentos inesperados e difíceis de corrigir. Para evitar esses problemas, é recomendável priorizar o uso de variáveis locais, passar argumentos explicitamente para funções e encapsular estados em classes, garantindo um código mais seguro, organizado e sustentável.

Uma curiosidade interessante sobre funções em Python é que elas são cidadãs de primeira classe (first-class citizens), o que significa que podem ser atribuídas a variáveis, passadas como argumentos para outras funções e até mesmo retornadas por outras funções. Isso permite a criação de funções anônimas (lambda) e a programação funcional com `map()`, `filter()` e `reduce()`, além de tornar o código mais flexível e reutilizável. Esse recurso é amplamente utilizado em frameworks modernos, como Flask e Django, nos quais as funções são passadas como *callbacks* para manipular requisições HTTP de forma dinâmica.

Considerações finais

O uso de funções e procedimentos é essencial para escrever códigos mais organizados, modulares e eficientes. Essas práticas facilitam a reutilização de código, a separação de responsabilidades e reduzem a complexidade dos sistemas. Além disso, a programação estruturada torna o código mais claro, o que simplifica sua manutenção, depuração e futuras melhorias, sem comprometer a lógica central do programa.

Outro ponto fundamental é o conceito de escopo de variáveis, que ajuda a evitar conflitos e erros inesperados, garantindo que cada variável seja usada no contexto certo. A distinção entre variáveis globais e locais é imprescindível para manter o código organizado, permitindo que funções e procedimentos operem de forma independente quando necessário.

No caso do Python, a linguagem oferece suporte a estruturas modulares e funções anônimas e permite a passagem de funções como

argumentos, tornando essa abordagem ainda mais poderosa. Esses conceitos são amplamente aplicados em frameworks modernos e no desenvolvimento de aplicações escaláveis, reforçando a importância de dominá-los desde o início da jornada na programação.

Ao adotar funções e procedimentos da maneira certa, os desenvolvedores conseguem escrever códigos mais eficientes, reutilizáveis e alinhados com as boas práticas da engenharia de software, criando soluções bem estruturadas e fáceis de evoluir.

Referências

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados com aplicações em Python. 4. ed. São Paulo: Bookman, 2022. *E-book*.

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Porto Alegre: Cengage Learning, 2022. *E-book*.

Listas

O computador e suas estruturas de suporte ao processamento de dados passaram por diversos aperfeiçoamentos ao longo do tempo, visando garantir o melhor uso de um recurso essencial para serviços e aplicações modernas: a memória. Com o crescimento exponencial da necessidade de processamento de informações, torna-se fundamental adotar estratégias eficientes para a gestão desse recurso. Para a resolução sofisticada de problemas computacionais, levando em consideração o uso otimizado e inteligente da memória, surge a área da computação denominada estruturas de dados (Forbellone; Eberspächer, 2022). Esta obra se dedica ao estudo e implementação de diferentes formas de organização e manipulação dos dados, permitindo não apenas um armazenamento eficiente, mas um acesso e processamento rápidos.

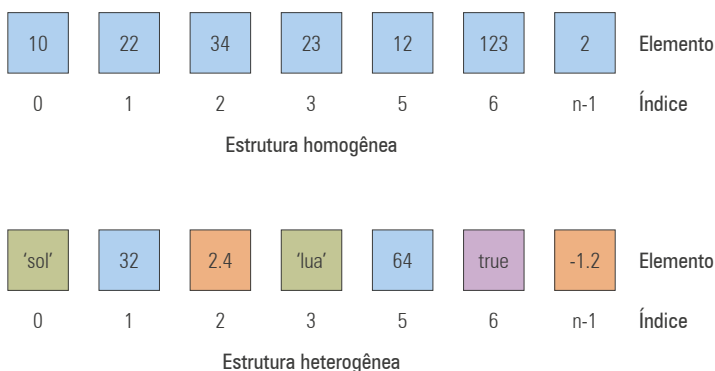
A maior parte das aplicações que lidam com grandes volumes de dados emprega estruturas de dados avançadas, garantindo um equilíbrio entre a eficiência da memória e o custo computacional adequado para cada problema (Lambert, 2022). A escolha correta dessas estruturas impacta diretamente o desempenho do software, tornando fundamental a compreensão de suas características e aplicabilidades. No contexto da programação, especialmente em linguagens de alto nível como Python, a implementação de estruturas de dados desempenha um papel essencial na criação de algoritmos eficientes. Python oferece uma ampla variedade de recursos para a manipulação de dados, proporcionando flexibilidade e desempenho, fatores que tornam essa linguagem uma das mais populares na atualidade (Guilhon *et al.*, 2022). Dessa forma, o estudo de estruturas de dados é essencial para qualquer programador ou cientista de dados, pois permite a criação de soluções mais otimizadas e escaláveis, alinhadas às necessidades computacionais modernas.

Neste capítulo será abordado o conceito de lista, uma das estruturas de dados mais utilizadas nas aplicações modernas, com muitas possibilidades para a resolução de problemas envolvendo práticas de programação. A primeira seção trata do conceito de estrutura unidimensional de dados, apresentando sua vantagem frente às variáveis simples e suas desvantagens quando submetida a um grande volume de dados. A segunda seção apresenta as operações básicas com listas, com exemplos práticos na linguagem de programação Python. A terceira seção aborda a correlação entre listas e funções, reforçando sua importância em operações específicas com listas. Por último, a quarta seção apresenta o conceito de estruturas bidimensionais e suas principais aplicações, com destaque para a representação matricial da matemática aplicada à computação.

1 Estruturas unidimensionais

As estruturas unidimensionais permitem o armazenamento, em tempo de execução, de conjuntos de dados de diferentes tipos, seja de maneira homogênea ou heterogênea. Enquanto o armazenamento homogêneo agrupa dados do mesmo tipo, o armazenamento heterogêneo agrupa dados de tipos diferentes. Independentemente das características dos dados, o armazenamento sempre terá uma abstração em uma única dimensão. É importante reforçar que esses tipos de armazenamento são encontrados na linguagem de programação Python. A figura 1 apresenta um exemplo de estrutura unidimensional e as diferentes organizações dos dados.

Figura 1 – Estruturas unidimensionais homogêneas e heterogêneas



Na figura 1, é possível observar que, independentemente do conjunto de dados, a estrutura está sempre encapsulada em um elemento referenciado por um índice, o qual, na maioria das linguagens de programação, inicia seu primeiro elemento em zero e tem no elemento final o índice com o número de elementos (representado por n na figura 1) subtraído em 1. Isso permite localizar facilmente um dado a partir de seu índice por meio de técnicas de programação ou certificar-se de que ele não existe na estrutura.

Outro aspecto das estruturas unidimensionais está relacionado ao uso da memória: trata-se da alocação dinâmica da memória ou uso da memória em bloco contíguo. A estrutura de lista em Python (list) utiliza a técnica de alocação dinâmica da memória, que possibilita um melhor aproveitamento dos espaços ociosos em memória, evitando fragmentações. Além disso, as listas possuem maior flexibilidade de operação com elementos, o que torna seu uso frequente nas práticas de programação modernas. A seguir, é apresentado um código-fonte com uso de listas nos formatos homogêneo e heterogêneo. Além do acesso a todos os dados da lista, o exemplo mostra como acessar um elemento específico:

```
# Lista homogênea de números inteiros
numeros = [10, 20, 30, 40, 50]

# Acessar o terceiro elemento (índice 2)
print("Lista homogênea:", numeros)
print("Terceiro elemento:", numeros[2])
```

Saída esperada:

Lista homogênea: [10, 20, 30, 40, 50]

Terceiro elemento: 30

```
# Lista heterogênea com diferentes tipos de dados
dados = ["sol", 32, 2.4, "lua", 64, True, -1.2]

# Acessar o quarto elemento (índice 3)
print("Lista heterogênea:", dados)
print("Quarto elemento:", dados[3])
```

Saída esperada

Lista heterogênea: ['sol', 32, 2.4, 'lua', 64, True, -1.2]

Quarto elemento: lua

O armazenamento em bloco contíguo faz a alocação sequencial de dados na memória, sendo comum em estruturas unidimensionais tradicionais de linguagens como C/C++. Apesar da possibilidade de fragmentação de memória (espaços ociosos resultantes do armazenamento sequencial), esse formato de alocação de dados é muito utilizado em aplicações voltadas para grandes volumes em cálculos matemáticos. Outra observação importante é que, na maioria das linguagens de programação, essa estrutura de dados é homogênea e chamada de vetor. Em Python, essa estrutura homogênea e contígua pode ser utilizada através da estrutura array, conforme apresentado a seguir.

```
import array

# Criar um array de números inteiros ('i' indica tipo inteiro)
numeros = array.array('i', [10, 20, 30, 40, 50])

# Exibir o array completo
print("Array:", numeros)

# Acessar o terceiro elemento (índice 2)
print("Terceiro elemento:", numeros[2])
```

Saída esperada:

Array: array('i', [10, 20, 30, 40, 50])

Terceiro elemento: 30



IMPORTANTE

As listas em Python são altamente flexíveis, permitindo armazenar elementos de diferentes tipos, crescer dinamicamente e suportar operações diversas sem a necessidade de especificar um tamanho fixo. No entanto, essa flexibilidade tem um custo em desempenho e uso de memória, pois as listas armazenam referências aos objetos em vez de valores diretamente. Já a estrutura array.array, embora limitada a um único tipo de dado, oferece um desempenho superior em operações matemáticas, pois armazena os elementos de forma contígua na memória, permitindo cálculos

mais rápidos e eficientes, além de consumir menos espaço. Para aplicações numéricas mais exigentes, bibliotecas como NumPy vão além, proporcionando maior otimização e suporte a operações vetorizadas.

2 Operações básicas

Depois de apresentado o conceito de estruturas unidimensionais, o próximo passo é entender as operações básicas nessas estruturas, concentrando-se na operação com listas. Serão apresentados exemplos de inserção, remoção, alteração e busca de elementos em uma lista. A estrutura de lista em Python possui diferentes funções que permitem uma grande lista de operações.

Um elemento pode ser inserido de várias maneiras em Python, sendo a mais tradicional a inserção ao final da lista pelo método `append()`, como apresentado no exemplo a seguir.

```
numeros = [10, 20, 30]
numeros.append(40) # Adiciona 40 no final
print(numeros)
```

O elemento também pode ser inserido em uma posição especificada pelo programador através da função `insert()`:

```
numeros = [10, 20, 30]
numeros.insert(1, 15) # Insere 15 na posição 1
print(numeros)
```

Também é possível inserir vários elementos ao final da lista por meio da função `extend()`:

```
numeros = [10, 20, 30]
numeros.extend([40, 50, 60]) # Adiciona múltiplos valores ao final
print(numeros)
```

Para a remoção de elementos de uma lista, também existem várias opções de funções. Por exemplo, a função `remove()` exclui um elemento encontrado em sua primeira ocorrência, realizando a busca do início ao fim da lista.

```
numeros = [10, 20, 30, 40, 30, 50]
numeros.remove(30) # Remove a primeira ocorrência de 30
print(numeros)
```

Outra opção é a remoção através do índice utilizando a função `del()`:

```
numeros = [10, 20, 30, 40, 50]
del numeros[1] # Remove o elemento no índice 1 (20)
print(numeros)
```

O programador também pode remover todos os elementos da lista com o comando `clear()`:

```
numeros = [10, 20, 30, 40, 50]
numeros.clear() # Esvazia a lista
print(numeros)
```

Para alterar o valor de um elemento em Python, é preciso conhecer o elemento que se deseja modificar. Com o índice do elemento identificado, basta atribuir-lhe um novo valor:

```
numeros = [10, 20, 30, 40, 50]

# Alterar o elemento no índice 2 (de 30 para 35)
numeros[2] = 35
print(numeros)
```

Independentemente da maneira que desejamos manipular uma lista, a busca de um elemento pode ser realizada de diversas formas: utilizando operações já implementadas nas listas em Python ou realizando uma busca personalizada através de loops.

Quanto às implementações em Python para a busca de elementos, uma maneira muito utilizada pela comunidade de especialistas em programação é o operador `in`, no qual se faz a busca por um valor existente:

```
numeros = [10, 20, 30, 40, 50]

# Verifica se 30 está na lista
if 30 in numeros:
    print("30 está na lista")
```

Todavia, a busca pode ser realizada de maneira personalizada por meio de loops, percorrendo elemento a elemento da lista. Para cada elemento percorrido, verifica-se se ele corresponde ao elemento-alvo. Caso seja encontrado, o especialista pode tomar a ação desejada:

```
numeros = [10, 20, 30, 40, 50]

# Buscar manualmente
for i in range(len(numeros)):
    if numeros[i] == 30:
        print("Encontrado no índice:", i)
        break
```

O código cria uma lista chamada *numeros* contendo os valores [10, 20, 30, 40, 50] e, em seguida, percorre essa lista utilizando um loop `for` que vai de 0 até `len(numeros) - 1` (o último índice da lista). A função `len()` faz a contagem do número de elementos existentes em uma lista. Dentro do loop, a estrutura `if numeros[i] == 30` verifica se o elemento no índice atual é igual a 30. Se for igual a 30, ele imprime a mensagem "Encontrado no índice:" seguida do índice correspondente, e usa o `break` para interromper o loop imediatamente após encontrar o primeiro 30, evitando verificações desnecessárias. Isso melhora a eficiência, pois o loop não continua executando depois que o elemento procurado é encontrado.



IMPORTANTE

Python oferece diversas possibilidades para a manipulação de listas, tornando essa estrutura de dados extremamente flexível e eficiente. É possível adicionar novos elementos tanto ao final quanto em posições específicas, além de combinar múltiplas coleções em uma única. Da mesma forma, a remoção de itens pode ser feita de diferentes maneiras, seja excluindo um valor específico, seja retirando elementos por posição ou esvaziando completamente a estrutura. Para localizar dados, existem abordagens que permitem verificar a existência de um item, identificar sua posição ou até encontrar todas as suas ocorrências. Além das operações básicas, há recursos avançados que possibilitam transformar listas de forma eficiente, ordenar valores automaticamente e aplicar funções para modificar os elementos de maneira otimizada. Com essa variedade de ferramentas, a manipulação de listas em Python se adapta a diversas necessidades, desde tarefas simples até o processamento de grandes conjuntos de dados.

3 Listas e funções

Nas linguagens de programação modernas, o desenvolvimento de funções e listas está totalmente integrado, tendo em vista as diferentes possibilidades de manipulação de dados.

Quando funções e listas são utilizadas em conjunto, é possível encontrar diferentes cenários, como o uso de listas como argumento de funções e o retorno de listas em funções.

No que se refere ao uso de listas como argumento de funções, em Python, as listas são passadas por referência. Assim, o que é passado não é uma cópia da lista, e sim uma referência de acesso à lista original, como apresenta o código a seguir:

```
def adicionar_elemento(lista):  
    lista.append(100) # Modifica a lista original
```

```
numeros = [10, 20, 30]
adicionar_elemento(numeros)

print("Lista após a função:", numeros)
```

Saída esperada:

Lista após a função: [10, 20, 30, 100]

É importante ressaltar a possibilidade de copiar a lista para evitar que ela seja manipulada em sua referência. A lista em Python possui o comando `copy()`, que permite desvincular a cópia da lista da referência anterior:

```
def adicionar_elemento_sem_modificar(lista):
    nova_lista = lista.copy() # Cria uma cópia
    nova_lista.append(100)
    return nova_lista # Retorna a lista modificada sem
    alterar a original

numeros = [10, 20, 30]
nova_lista = adicionar_elemento_sem_modificar(numeros)

print("Lista original:", numeros)
print("Nova lista modificada:", nova_lista)
```

No que diz respeito ao retorno de valores dessa lista, é necessário ter cuidado, pois o retorno da referência de uma lista ou a cópia de uma nova lista podem trazer mudanças não desejadas à lista original. A seguir, apresentamos um exemplo do retorno da referência de uma lista sem a realização de uma cópia.

```
def adicionar_elemento(lista):
    lista.append(100) # Modifica a lista original
    return lista # Retorna a referência da lista original

numeros = [10, 20, 30]
nova_lista = adicionar_elemento(numeros)
```

```
print("Lista original:", numeros) # Foi modificada
print("Lista retornada:", nova_lista) # Mesmo endereço na memória
```

Saída esperada

Lista original: [10, 20, 30, 100]

Lista retornada: [10, 20, 30, 100]

O programador pode optar pelo retorno da cópia de uma lista, realizando as modificações que desejar sem afetar a lista original:

```
def adicionar_elemento_sem_modificar(lista):
    nova_lista = lista.copy() # Cria uma cópia da lista original
    nova_lista.append(100) # Modifica apenas a cópia
    return nova_lista # Retorna a cópia modificada

numeros = [10, 20, 30]
nova_lista = adicionar_elemento_sem_modificar(numeros)

print("Lista original:", numeros) # Permanece inalterada
print("Lista retornada:", nova_lista) # Cópia com modificação
```

Saída esperada

Lista original: [10, 20, 30]

Lista retornada: [10, 20, 30, 100]

4 Estruturas bidimensionais

Além das estruturas unidimensionais nas estruturas de dados elementares, também devem ser consideradas as estruturas bidimensionais, que organizam os dados em duas dimensões: linhas e colunas. Linguagens de programação como C/C++ e Java possuem estruturas nativas de representação bidimensional através de matrizes. Em Python,

a representação de matrizes é adaptada por meio do uso de listas aninhadas. No exemplo a seguir, listas nativas são utilizadas para representar uma matriz de três linhas e três colunas:

```
# Criando uma matriz 3x3 usando listas aninhadas
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Acessando um elemento específico (linha 1, coluna 2)
print(matriz[1][2]) # Saída: 6
```

Também é importante reforçar que percorrer dados em estruturas bidimensionais exige o uso de dois laços de repetição: um para percorrer a primeira dimensão e outro, em cascata, para controlar a segunda dimensão:

```
# Definição de uma matriz 3x3
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Percorrendo a matriz linha por linha
for linha in matriz:
    for elemento in linha:
        print(elemento, end=" ")
    print() # Nova linha para organizar a saída
```

Saída esperada

1 2 3

4 5 6

7 8 9



IMPORTANTE

Na programação, as matrizes são fundamentais para representar e manipular operações matemáticas de forma eficiente, permitindo resolver problemas complexos em diversas áreas, como álgebra linear, computação gráfica, inteligência artificial e estatística. Ao organizar os dados em uma estrutura bidimensional, é possível realizar cálculos matriciais, como soma, multiplicação, inversão e determinantes, de maneira sistemática e otimizada. Em linguagens como Python, bibliotecas como NumPy oferecem suporte avançado para operações vetorizadas, reduzindo o tempo de processamento e tornando os cálculos mais rápidos e precisos. Além disso, matrizes são essenciais na representação de sistemas de equações lineares, transformações geométricas e processamento de imagens, tornando-se uma ferramenta indispensável para aplicações científicas e computacionais.

Considerações finais

As listas são uma das estruturas de dados mais versáteis e amplamente utilizadas na programação, desempenhando um papel essencial na organização e manipulação de conjuntos de informações. Neste capítulo, foram exploradas suas principais características, desde o armazenamento unidimensional e bidimensional até as operações básicas de inserção, remoção, busca e alteração de elementos. Além disso, foi demonstrada a relação entre listas e funções, destacando a importância da passagem por referência e cópia para evitar modificações não desejadas nos dados originais. Também foi abordada a eficiência de diferentes abordagens para a manipulação de listas, incluindo o uso de estruturas como array, que oferecem melhor desempenho em operações matemáticas e computacionais.

Compreender essas técnicas é fundamental para o desenvolvimento de programas mais eficientes, organizados e escaláveis, permitindo que

os programadores aproveitem ao máximo os recursos disponíveis na linguagem Python. Ao dominar esses conceitos, é possível resolver problemas de forma mais estruturada, aplicando boas práticas de programação e facilitando a manutenção do código.

Referências

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados com aplicações em Python. 4. ed. São Paulo: Bookman, 2022. *E-book*.

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Porto Alegre: Cengage Learning, 2022. *E-book*.

Fatiamento (slicing)

Manipular estruturas de dados de forma prática é um diferencial para qualquer profissional de linguagem de programação. Segundo Lambert (2022), a linguagem Python oferece recursos avançados para manipulação eficiente de dados, incluindo o fatiamento (slicing), uma técnica poderosa para refinar e extrair subconjuntos de elementos em listas. Essa abordagem permite maior flexibilidade e otimização no processamento de informações, tornando-se uma ferramenta essencial para programadores que buscam proporcionar eficiência e clareza ao código. A prática de programação com a aplicação adequada de fatiamento acelera o processo de desenvolvimento e gera um custo computacional menor frente a outros métodos baseados em ausência de força bruta (por exemplo, o loops).

O primeiro subcapítulo trata de fatiamento de listas no geral, apresentando a sintaxe básica e, principalmente, a mentalidade essencial para realizar o fatiamento desejado. No próximo subcapítulo, é abordada a manipulação de strings – lembrando que essa manipulação, na linguagem de programação Python, trata-se de considerar uma lista de caracteres em que as técnicas de fatiamento se tornam bem-vindas para que se realizem edições.

1 Fatiamento de listas

O fatiamento de listas é uma técnica em Python que permite acessar, modificar e manipular subconjuntos de elementos dentro de uma lista de maneira eficiente e expressiva. Utilizando uma sintaxe intuitiva, o fatiamento oferece uma maneira poderosa de interagir com estruturas de dados sequenciais, evitando a necessidade de loops explícitos e tornando o código mais legível e com melhor desempenho. A sintaxe básica para a aplicação de fatiamento é:

```
lista[início:fim:passo]
```

Em que:

- *início* é o índice do primeiro elemento a ser incluído no fatiamento (opcional; padrão é 0).
- *fim* é o índice do primeiro elemento a ser excluído do fatiamento (opcional; padrão é o tamanho da lista).
- *passo* define a variação entre os índices extraídos (opcional; padrão é 1).

Um primeiro exemplo é a extração de uma sublista a partir de uma lista de origem, conforme a seguir.

Exemplo 1

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sublista = numeros[2:7] # Do índice 2 ao 6
print(sublista) # Saída: [2, 3, 4, 5, 6]
```

É importante observar que a regra do fatiamento consiste em que o primeiro parâmetro diz respeito ao primeiro elemento a ser incluído no fatiamento, e o segundo parâmetro seria o primeiro elemento a ser excluído do fatiamento; sempre da esquerda para a direita.

Portanto, no exemplo 1, o primeiro parâmetro indica que o índice 2 é o primeiro a ser incluído (logo, os índices 0 e 1 estão excluídos do fatiamento). Por sua vez, o índice 7 é o segundo parâmetro, sendo o primeiro a ser excluído (logo, os índices 7, 8 e 9 estão excluídos do fatiamento). O resultado do fatiamento é a sublista [2, 3, 4, 5, 6].

Outro aspecto importante é quando o parâmetro do passo é utilizado, permitindo uma seleção alternada e controlada dos elementos da lista, como mostra o exemplo 2.

Exemplo 2

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
pares = numeros[::2] # Pegando elementos com passo 2
print(pares) # Saída: [0, 2, 4, 6, 8]
```

No exemplo 2, os parâmetros de início e fim estão ajustados para seu funcionamento padrão: o primeiro elemento a ser incluído é o índice zero, e o primeiro elemento a ser excluído da lista é referente ao tamanho da lista; portanto, toda a lista está sendo contemplada. Porém, o parâmetro de passo 2 faz a coleta alternada partindo do primeiro elemento do resultado do fatiamento, gerando a saída apresentada no exemplo 2.

No fatiamento, também é possível trabalhar com o passo negativo, o que resulta na coleta inversa de elementos, partindo do primeiro elemento resultante do fatiamento, como apresentado no próximo exemplo (exemplo 3).

Exemplo 3

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
invertida = numeros[::-1] # Passo negativo para inverter
print(invertida) # Saída: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Como já abordado, os valores padrão dos dois primeiros parâmetros do fatiamento resultam em coletar toda a lista. O passo com valor negativo -1 resulta na coleta inversa dos valores (do maior índice resultante

do fatiamento para o menor), se constituindo em uma técnica simples para inverter elementos de listas.

Outra funcionalidade interessante é a modificação de partes de uma lista, substituindo elementos de forma direta, como apresentado exemplo 4.

Exemplo 4

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
numeros[2:5] = [20, 30, 40] # Substituindo elementos
print(numeros) # Saída: [0, 1, 20, 30, 40, 5, 6, 7, 8, 9]
```

A partir do fatiamento realizado (incluindo os elementos a partir do índice 2 e excluindo os elementos a partir do índice 5), podemos realocar uma outra lista com o mesmo número de elementos; neste caso, 3 elementos. Mesmo que alocados dinamicamente, a operação de substituição é realizada internamente pelo interpretador Python.

Além da substituição, o segmento de lista selecionado pode ser removido por meio do comando `del`, conforme o exemplo 5 a seguir.

Exemplo 5

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
del numeros[2:5] # Remove os elementos do índice 2 ao 4
print(numeros) # Saída: [0, 1, 5, 6, 7, 8, 9]
```

No exemplo 5, o segmento selecionado é extraído, gerando uma lista com o resultado da junção dos elementos não contemplados no fatiamento.

Em resumo, o uso do fatiamento traz diversos benefícios, incluindo:

- Legibilidade: a sintaxe é concisa e fácil de entender.
- Eficiência: opera diretamente na lista, evitando loops desnecessários.

- Flexibilidade: permite realizar extrações, substituições e manipulações complexas de forma simples.



IMPORTANTE

É fundamental ter em mente que, ao utilizar o fatiamento para extrair partes de uma lista em Python, o resultado será sempre uma nova lista, completamente independente da original. Esse comportamento pode ter implicações significativas no consumo de memória, especialmente ao manipular grandes volumes de dados, pois cada operação de fatiamento cria uma cópia dos elementos selecionados. Consequentemente, em cenários que envolvem listas extensas, é importante avaliar alternativas mais eficientes, como iteradores ou views, para evitar o consumo desnecessário de memória e melhorar o desempenho geral do programa.

2 Operações com strings

As strings são uma das estruturas de dados mais fundamentais em Python. Elas representam sequências de caracteres e são imutáveis; ou seja, uma vez criadas, não podem ser modificadas diretamente. No entanto, Python oferece diversas operações para manipular strings de forma eficiente. Entre essas operações, destacam-se concatenação, repetição, acesso por índice, fatiamento e diversas funções embutidas para modificação e análise de texto.

A concatenação de strings em Python é realizada por meio do operador +, que permite unir duas ou mais sequências de caracteres em uma única string. Essa operação é útil para a construção dinâmica de textos, como mostra o exemplo 6 a seguir.

Exemplo 6

```
s1 = "Olá"
s2 = "Mundo"
resultado = s1 + " " + s2
print(resultado) # Saída: Olá Mundo
```

No exemplo 6, as strings "Olá" e "Mundo" foram unidas, com um espaço intermediário para melhor legibilidade.

O operador `*` permite a repetição de uma string por um determinado número de vezes. Essa técnica é útil para criar padrões de texto que se repetem, conforme apresenta o exemplo 7.

Exemplo 7

```
s = "Python "
print(s * 3) # Saída: Python Python Python
```

Também existem funções embutidas que apoiam sua manipulação. Por exemplo, a função `len()` retorna o número de caracteres presentes em uma string, como mostra o exemplo a seguir.

Exemplo 8

```
s = "Python"
print(len(s)) # Saída: 6
```

Um caractere específico pode ser acessado por seu índice. Em Python, cada caractere de uma string pode ser acessado individualmente por meio de índices, cuja contagem começa do zero. Índices negativos permitem a contagem a partir do final da string. Dessa forma, podemos acessar tanto os primeiros caracteres quanto os últimos de forma simples.

Exemplo 9

```
s = "Python"
print(s[0]) # Saída: P (primeiro caractere)
print(s[-1]) # Saída: n (último caractere)
```

No entanto, referente ao fatiamento de strings, as regras são as mesmas aplicadas às listas, utilizando-se a sintaxe `string[início:fim:passo]`. O índice de início é inclusivo, enquanto o índice de fim é exclusivo. A seguir, alguns exemplos.

Exemplo 10

```
s = "Python"
print(s[0:4]) # Saída: Pyth (pega do índice 0 ao 3)
print(s[:4]) # Saída: Pyth (começa no início por padrão)
print(s[2:]) # Saída: thon (vai do índice 2 até o final)
print(s[-4:]) # Saída: thon (pega os últimos quatro caracteres)
print(s[::2]) # Saída: Pto (pula de 2 em 2)
```

Seguindo as mesmas premissas de fatiamento de strings, podemos, por exemplo, inverter a ordem de uma string (essa técnica é frequentemente utilizada para verificar palíndromos). Vejamos mais um exemplo.

Exemplo 11

```
s = "Python"
print(s[::-1]) # Saída: nohtyP
```

A linguagem Python também oferece operadores de verificação de um dado ou parte de um dado em uma string através dos operadores `in` (retorna valor "verdadeiro" se a substring estiver contida na string analisada) e `not in` (retorna valor "falso" se a substring estiver contida na string analisada). Veja o exemplo 12 a seguir.

Exemplo 12

```
s = "Python é incrível"
print("Python" in s) # True
print("Java" not in s) # True
```

Considerações finais

Neste capítulo, exploramos a técnica de fatiamento (slicing) em Python, aplicada tanto a listas quanto a strings. Vimos como essa funcionalidade permite acessar, modificar e manipular subconjuntos de elementos de forma eficiente, reduzindo a necessidade de loops e tornando o código mais legível. No caso das listas, abordamos a sintaxe básica do fatiamento `lista[início:fim:passo]` e a utilização de passos positivos e negativos, além de operações como substituição e remoção de elementos. Já nas strings, destacamos a aplicação do slicing para extração de trechos, inversão de caracteres e verificação de substrings. A partir desses conceitos, o leitor agora dispõe de uma base sólida para manipular dados de maneira otimizada em Python, em que é possível aproveitar ao máximo o poder do fatiamento para tornar o código mais conciso e com melhor desempenho.

Referências

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. São Paulo: Cengage Learning Brasil, 2022. *E-book*.

Tuplas e dicionários

As linguagens de programação possuem estruturas poderosas para abstrações de dados essenciais ao desenvolvimento de aplicações robustas e adequadas ao contexto de uso. Em Python, não é diferente: a linguagem oferece diversas estruturas que permitem adaptar soluções de tecnologia da informação às necessidades específicas do projeto. Conforme discutido por Forbellone e Eberspächer (2022), a organização e a manipulação eficiente de dados são fundamentais para a construção de algoritmos bem-estruturados, sendo as tuplas e os dicionários duas das principais ferramentas para esse fim.

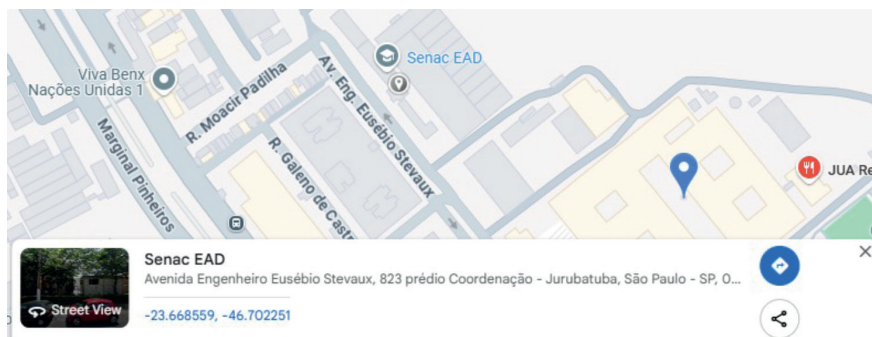
Neste capítulo, serão abordadas as estruturas de tuplas e dicionários, comuns em linguagens multifuncionais, como a Python, pois oferecem diferentes formas de representação de dados e flexibilidade em sua manipulação, permitindo a adaptação a distintos contextos de desenvolvimento (Guilhon *et al.*, 2022). No primeiro subcapítulo, será tratado o conceito de tupla, enfatizando sua característica posicional e imutável. Em seguida, serão abordados o uso de dicionários e sua capacidade de armazenar dados de maneira rotulada, facilitando o processo de

recuperação e organização, como pontua Lambert (2022). Por fim, serão discutidas situações específicas do uso combinado de dicionários e tuplas em conjunto com funções, demonstrando sua aplicabilidade prática na construção de soluções eficientes.

1 Tuplas

No mundo da computação e da matemática, temos algumas representações para as quais a ordem e a imutabilidade são garantias para a confiabilidade de um dado. Um primeiro exemplo são os pontos de sistemas cartesianos (bidimensionais ou tridimensionais), aplicados nas diferentes representações de dados. A figura 1 mostra um exemplo de aplicação da representação da latitude e longitude em um mapa, na qual é imprescindível a ordem de apresentação dos dados na estrutura.

Figura 1 – Exemplo do uso de coordenadas geográficas em um mapa



Outro exemplo que envolve nosso cotidiano está relacionado aos sistemas de cores utilizados no meios digitais e impressos. No meio digital, o sistema de cor utilizado é o RGB (representação das cores red, green e blue – vermelho, verde e azul), e no meio impresso é utilizado o padrão CMKY (representação das cores cyan, magenta, yellow e key – ciano, magenta, amarelo e preto). É importante também reforçar que a ordem de representação dos dados também é imprescindível. O quadro 1 apresenta alguns exemplos de combinações do padrão RGB e CYMK.

Quadro 1 – Exemplo da representação do padrão RGB e CYMK

COR	RGB	CMYK
Vermelho	RGB(255, 0, 0)	CMYK(0, 100, 100, 0)
Verde	RGB(0, 255, 0)	CMYK(100, 0, 100, 0)
Azul	RGB(0, 0, 255)	CMYK(100, 100, 0, 0)
Amarelo	RGB(255, 255, 0)	CMYK(0, 0, 100, 0)
Ciano	RGB(0, 255, 255)	CMYK(100, 0, 0, 0)
Magenta	RGB(255, 0, 255)	CMYK(0, 100, 0, 0)
Branco	RGB(255, 255, 255)	CMYK(0, 0, 0, 0)
Preto	RGB(0, 0, 0)	CMYK(0, 0, 0, 100)
Cinza	RGB(128, 128, 128)	CMYK(0, 0, 0, 50)

Na linguagem de programação Python, as tuplas são estruturas que melhor representam os exemplos apresentados, dada sua imutabilidade; ou seja, *os dados não podem ser alterados depois de sua criação*. Para criar um tupla em Python, a seguinte sintaxe deve ser utilizada:

```
minha_tupla = (1, 2, 3)
```

Em que cada elemento da tupla deve ser atribuído na ordem adequada para o futuro tratamento dos dados.

Corroborando com nossos exemplos, podemos representar coordenadas geográficas e cores utilizando tuplas, conforme segue.

```
#coordenada geográfica
lat_long_senac = (-23.668559, -46.702251)
#rgb
vermelho_rgb = (255, 0, 0)
#cmky
amarelo_cmyk = (0, 0, 100, 0)
```

Para acessar o elemento de uma tupla, basta utilizar seu índice, começando por zero até o n-ésimo elemento menos 1. Por exemplo: se a tupla possui 3 elementos, os índices serão rotulados de 0 a 2. Observe o exemplo 1.

Exemplo 1

```
print(vermelho_rgb[0]) # Saída: 255
print(amarelo_cmyk[2]) # Saída: 100
```

Outro ponto importante é entender que tuplas também permitem diferentes tipos de dados, como mostra o exemplo 2.

Exemplo 2

```
info_cor = ("Vermelho", (255, 0, 0), "#FF0000", True)
print(info_cor)
# Saída: ('Vermelho', (255, 0, 0), '#FF0000', True)
```

No exemplo 2, "Vermelho" é uma string, (255, 0, 0) é uma tupla aninhada representando RGB, '#FF0000' é a representação hexadecimal da cor e True indica que essa cor é considerada "quente".

As tuplas em Python possuem uma opção de desempacotamento, o qual seria o processo simplificado de atribuição dos dados da tupla a variáveis individuais. Observe o exemplo 3.

Exemplo 3

```
r, g, b = vermelho_rgb
print(f"R: {r}, G: {g}, B: {b}") # Saída: R: 255, G: 0, B: 0
```

Por fim, é importante reforçar que a imutabilidade de tuplas em Python é uma premissa da linguagem; ou seja, caso seja feita uma tentativa de armazenar um novo valor em um elemento de uma tupla, um erro será gerado.

```
vermelho_rgb[0] = 200 # Isso resultará em um erro!
```



IMPORTANTE

As tuplas desempenham um papel fundamental na matemática e na computação em Python, pois oferecem uma maneira eficiente e estruturada de representar conjuntos imutáveis de valores. Sua imutabilidade garante segurança em operações que exigem dados fixos, como coordenadas cartesianas, vetores, pontos em gráficos, constantes físicas e representações de cores nos modelos RGB e CMYK, em que cada cor pode ser armazenada como uma tupla de três ou quatro valores, respectivamente. Além disso, a capacidade das tuplas de armazenar diferentes tipos de dados as torna ideais para representar estruturas heterogêneas, como registros de banco de dados, informações de configuração e agrupamentos de valores matemáticos em álgebra linear e geometria computacional. Seu desempenho superior em relação às listas, devido à otimização interna de armazenamento, faz das tuplas uma escolha eficiente para aplicações que exigem integridade dos dados e processamento rápido.

2 Dicionários

As linguagens de programação, ao longo de sua evolução, se adaptaram às necessidades de flexibilização no tratamento de dados de maneira a melhorar a produtividade na proposição de problemas na tecnologia da informação. Principalmente com o advento de padrões de permitem o intercâmbio de dados tais como o JSON e o XML, foram necessárias adaptações às linguagens de programação para trabalhar com esses tipos de dados com maior facilidade. As figuras 2 e 3 mostram os dados de uma pessoa em formato JSON e XML, respectivamente.

Figura 2 – Dados pessoais em formato JSON

```
{
  "pessoa": {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com",
    "telefone": "+55 11 99999-8888",
    "endereco": {
      "rua": "Av. Paulista",
      "numero": 1234,
      "cidade": "São Paulo",
      "estado": "SP",
      "cep": "01311-200"
    },
    "documentos": {
      "cpf": "123.456.789-00",
      "rg": "12.345.678-9"
    }
  }
}
```

Figura 3 – Dados pessoais em formato XML

```
<pessoa>
  <nome>João da Silva</nome>
  <idade>35</idade>
  <email>joao.silva@email.com</email>
  <telefone>+55 11 99999-8888</telefone>
  <endereco>
    <rua>Av. Paulista</rua>
    <numero>1234</numero>
    <cidade>São Paulo</cidade>
    <estado>SP</estado>
    <cep>01311-200</cep>
  </endereco>
  <documentos>
    <cpf>123.456.789-00</cpf>
    <rg>12.345.678-9</rg>
  </documentos>
</pessoa>
```

Contudo, podem também existir outros formatos específicos além do JSON e XML que permitem esta flexibilidade de porte de diferentes tipos de dados. Nesse caso, como a linguagem de programação pode trabalhar com a manipulação desses dados como uma estrutura única? A linguagem de programação Python soluciona esse tratamento através dos dicionários, sendo estruturas que têm por característica o armazenamento de dados em padrão de chave-valor. Eles permitem acesso rápido aos valores por meio de suas chaves, funcionando de forma semelhante a um mapa ou uma tabela de consulta. Para criar um dicionário, a sintaxe básica consiste no uso do `{}` ou a função `dict()`, como apresentado no exemplo 4.

Exemplo 4

```
# Criando um dicionário de uma pessoa
pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com",
    "telefone": "+55 11 99999-8888"
}

print(pessoa)
#saida
# {'nome': 'João da Silva', 'idade': 35, 'email': 'joao.silva@email.com', 'telefone': '+55 11 99999-8888'}
```

Nos exemplos das figuras 2 e 3, os respectivos dados podem ser representados em um dicionário em Python da seguinte maneira.

Exemplo 5

```
pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "email": "joao.silva@email.com",
    "telefone": "+55 11 99999-8888",
    "endereco": {
```

```

        "rua": "Av. Paulista",
        "numero": 1234,
        "cidade": "São Paulo",
        "estado": "SP",
        "cep": "01311-200"
    },
    "documentos": {
        "cpf": "123.456.789-00",
        "rg": "12.345.678-9"
    }
}

# Exemplo de acesso aos dados
print(pessoa["nome"]) # Saída: João da Silva
print(pessoa["endereço"]["cidade"]) # Saída: São Paulo
print(pessoa["documentos"]["cpf"]) # Saída: 123.456.789-00

```

É importante observar ao final do exemplo 5 o acesso aos valores de um dicionário, bastando informar a chave para receber um valor desejado. Outro ponto de observação é que dicionários também podem armazenar outros dicionários (dados de endereço e dados de documentos). Para acessar esses dados, basta mencionar as chaves em ordem de profundidade de acesso sempre utilizando `[]`.

Dicionários possuem elementos mutáveis, ou seja, passíveis de modificação ao longo de seu uso. A seguir, são apresentados exemplos de atribuição de novos dados a um dicionário, bastando informar a chave para a localização exata do ponto de alteração.

Exemplo 6

```

pessoa["idade"] = 36 # Atualizando um valor existente
pessoa["cidade"] = "São Paulo" # Adicionando uma nova chave

```

Para remover itens de um dicionário, deve ser utilizado o comando `del`, especificando a chave que pretende ser removida, como apresentam os exemplos a seguir. Outra opção apresentada é a remoção de uma chave e o retorno do valor através do comando `pop`.

Exemplo 7

```
del pessoa["telefone"] # Remove uma chave específica
print(pessoa)

telefone = pessoa.pop("email") # Remove e retorna o valor
associado
print(telefone) # Saída: joao.silva@email.com
```

Outro aspecto importante no que diz respeito aos dicionários é a técnica de iteração sobre os elementos. Para cada iteração, é acessado o item no qual uma tupla pode ser acessada, na qual o primeiro elemento é a chave e o segundo elemento é o valor, conforme o exemplo 8 a seguir.

Exemplo 8

```
# Percorrendo chaves e valores
for chave, valor in pessoa.items():
    print(f"{chave}: {valor}")
```

O dicionário possui em sua estrutura métodos nativos (funções implementadas para a estrutura específica) que facilitam a manipulação, tal como os métodos `keys()` (retorna todas as chaves), `values()` (retornam todos os valores) e `items()` (como apresentado no exemplo 9, retorna todos os itens).

Exemplo 9

```
print(pessoa.keys()) # Retorna todas as chaves
print(pessoa.values()) # Retorna todos os valores
print(pessoa.items()) # Retorna pares chave-valor
```

Uma opção importante que possibilita à pessoa desenvolvedora otimizar seu tempo é o método de verificação da existência de uma chave através do método `in`, como segue. O código apresentado no exemplo 10 verifica se a chave `email` está presente no dicionário `pessoa`.

Exemplo 10

```
if "email" in pessoa:
    print("E-mail cadastrado:", pessoa["contato"]["email"])
```



IMPORTANTE

Os dicionários em Python desempenham um papel crucial na manipulação de dados intercambiáveis, como aqueles representados em JSON e XML, facilitando a comunicação entre sistemas, APIs e bancos de dados. Sua estrutura baseada em pares chave-valor permite acesso eficiente às informações, tornando-os ideais para converter, armazenar e processar dados estruturados de forma dinâmica e flexível. Como o JSON é nativamente compatível com dicionários, a conversão entre esses formatos é simples, permitindo que aplicações web e serviços distribuídos troquem informações de maneira padronizada. Além disso, a manipulação de XML pode ser feita de forma eficiente utilizando dicionários, permitindo extrair, modificar e serializar dados para diversas aplicações. Essa capacidade torna os dicionários essenciais para o desenvolvimento de software moderno, garantindo interoperabilidade, organização e eficiência na manipulação de grandes volumes de dados.

3 Tuplas e dicionários em funções

As funções em Python podem receber tuplas e dicionários como argumentos e podem retorná-los como resultado. Isso é útil para manipular múltiplos valores de forma eficiente e organizada.

Tuplas são úteis quando uma função precisa receber um conjunto fixo de valores que não devem ser modificados. Perceba no exemplo 11 a passagem da tupla como argumento de uma função, destacando que ela tem a garantia de que os dados não podem ser modificados e consequentemente a função não pode modificar seus valores internos.

Exemplo 11

```
def exibir_dados_pessoais(dados):
    nome, idade, cidade = dados # Desempacotamento da tupla
    print(f"Nome: {nome}, Idade: {idade}, Cidade: {cidade}")

# Criando uma tupla com dados
pessoa = ("João da Silva", 35, "São Paulo")

# Passando a tupla como argumento
exibir_dados_pessoais(pessoa)

# saída
# Nome: João da Silva, Idade: 35, Cidade: São Paulo
```

Funções também pode retornar tuplas em casos em que seja necessário o retorno de um conjunto de valores em uma única estrutura imutável sem o uso de listas ou dicionários, como mostra o exemplo 12.

Exemplo 12

```
def calcular_idade_futura(idade_atual, anos):
    idade_futura = idade_atual + anos
    return idade_atual, idade_futura # Retorno como tupla

idade_atual, idade_futura = calcular_idade_futura(35, 5)
print(f"Idade atual: {idade_atual}, Idade em 5 anos: {idade_futura}")

#saída
# Idade atual: 35, Idade em 5 anos: 40
```

Quando se trata do uso de dicionários na passagem de valores ou retornos em funções, devem ser tomados os devidos cuidados, principalmente pela flexibilidade da estrutura. A passagem de um dicionário como um argumento de uma função é nativamente por referência, ou seja, a estrutura passada é manipulada na origem em que foi criada, sem a realização de uma cópia. O código-fonte, no exemplo 13 a seguir, mostra o uso de uma função que facilita a visualização dos dados de um dicionário.

Exemplo 13

```
def exibir_dados_pessoais(dados):
    print(f"Nome: {dados['nome']}, Idade: {dados['idade']},
    Cidade: {dados['cidade']}")

pessoa = {
    "nome": "João da Silva",
    "idade": 35,
    "cidade": "São Paulo"
}

exibir_dados_pessoais(pessoa)
#saida
#Nome: João da Silva, Idade: 35, Cidade: São Paulo
```

Reforçando a mutabilidade dos dicionários, o exemplo abaixo faz a inclusão de um par-chave e valor em um dicionário passado como argumento de uma função. Observe que esse dicionário é modificado em sua origem, como demonstra o exemplo 14. A função modificou diretamente o dicionário original, pois foi passado por referência.

Exemplo 14

```
def adicionar_email(pessoa):
    pessoa["email"] = "joao.silva@email.com"

dados_pessoais = {"nome": "João da Silva", "idade": 35}
adicionar_email(dados_pessoais)

print(dados_pessoais)

#saida
# {'nome': 'João da Silva', 'idade': 35, 'email': 'joao.silva@email.com'}
```

Quando a intenção não é a manipulação do dicionário por referência e, sim, ter uma cópia para ao final retornar um novo dicionário, é importante o uso do comando `copy()`. O exemplo 15 faz uma cópia simples de

um dicionário e retorna um novo, desvinculando o novo dicionário da origem do anterior.

Exemplo 15

```
def adicionar_email(pessoa):  
    nova_pessoa = pessoa.copy()  
    nova_pessoa["email"] = "joao.silva@email.com"  
    return nova_pessoa  
  
dados_pessoais = {"nome": "João da Silva", "idade": 35}  
dados_modificados = adicionar_email(dados_pessoais)  
  
print(dados_pessoais) # Original não é alterado  
print(dados_modificados) # Novo dicionário com a modificação
```



PARA SABER MAIS

Existem mais funcionalidades dos dicionários que podem ser exploradas! O `deepcopy()`, presente no módulo `copy`, é utilizado para criar cópias independentes de objetos complexos, garantindo que estruturas aninhadas sejam replicadas sem manter referências ao original, evitando modificações indesejadas. Já o `**kwargs` permite que funções recebam um número variável de argumentos nomeados em formato de dicionário, tornando-as mais flexíveis e reutilizáveis. Você pode pesquisar por ambas as funcionalidades em sua ferramenta de busca para aprofundar ainda mais seus estudos sobre dicionários.

Considerações finais

Neste capítulo, exploramos as tuplas e os dicionários como estruturas fundamentais da linguagem Python, destacando as características, os usos e as vantagens na manipulação de dados. As tuplas, com sua imutabilidade e organização posicional, se mostraram ideais para representar dados fixos, como coordenadas e modelos de cores. Já os

dicionários, por serem estruturas mutáveis e baseadas em pares chave-valor, se destacaram pela flexibilidade no armazenamento e recuperação de informações, sendo amplamente utilizados na manipulação de formatos como JSON e XML. Além disso, abordamos a interação dessas estruturas com funções, demonstrando a passagem de argumentos e o retorno de valores, bem como os impactos da imutabilidade das tuplas e da passagem por referência dos dicionários.

Referências

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados com aplicações em Python. 4. ed. São Paulo: Bookman, 2022. *E-book*.

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.

Função lambda

Nas práticas de programação, eficiência e clareza são essenciais, principalmente ao manipular grandes volumes de dados. Segundo Guilhon *et al.* (2022), Python se destaca nesse contexto por oferecer recursos que tornam o código mais enxuto e expressivo, favorecendo a legibilidade e a manutenção. Um desses recursos é a função lambda, que permite simplificar operações curtas sem a necessidade de criar funções nomeadas tradicionais. Como aponta Lambert (2022), compreender a sintaxe e a aplicação dessas funções é fundamental para aproveitar seu potencial na manipulação de dados, especialmente quando combinadas com ferramentas como `sort`, `map` e `list`, que ampliam sua eficiência e versatilidade.

Neste capítulo, exploraremos o conceito e as aplicações das funções lambda, mostrando como elas podem tornar o código mais compacto e legível. Na primeira seção, abordaremos a sintaxe e alguns usos práticos. Em seguida, veremos como combiná-las com funções como `sort`, `map` e `list`, ampliando o potencial de manipulação eficiente de dados.

1 Conceito e aplicações

Desenvolver aplicações e exercer o ato de programar consistem em prática constante de exercícios e métodos que resultam em resolver problemas de maneira eficiente e no menor tempo possível. Quando desenvolvemos aplicações em Python, organizamos nossos códigos em funções para um melhor aproveitamento, assim como aplicamos ações de refatoração. No entanto, podemos nos deparar com situações em que funções resolvem expressões curtas e simples, principalmente associadas a funções aritméticas ou de fatiamento de dados. Para essas situações, o Python possibilita o desenvolvimento de subprogramas em única linha, intitulada função lambda.

A função lambda recebe esse nome devido ao lambda calculus (cálculo lambda), um sistema matemático formal desenvolvido por Alonzo Church na década de 1930. Esse cálculo é um modelo de computação baseado em funções anônimas e expressões matemáticas puras, sem a necessidade de atribuições ou estados mutáveis.

Para escrever uma função lambda em Python, utiliza-se seguinte estrutura:

```
lambda parâmetros: expressão
```

Na qual *expressão* deve ter uma única linha e gerar o valor de retorno, e os *parâmetros*, separados entre vírgulas, para entradas de dados na expressão. Por exemplo: podemos ter uma função tradicional escrita conforme o exemplo 1:

Exemplo 1

```
def soma(x, y):  
    return x + y  
print(soma(3, 4)) # Saída: 7
```

Nesse caso, podemos montar uma função lambda equivalente da seguinte maneira:

```
soma = lambda x, y: x + y  
print(soma(3, 4)) # Saída: 7
```

É importante ressaltar que funções definidas com `def` podem ter múltiplas linhas e comentários e são mais legíveis. Em contrapartida, funções lambda são úteis para funções simples e descartáveis, usadas dentro de outras expressões.

Em situações em que a função é curta e utilizada apenas temporariamente, a aplicação de funções anônimas, como as funções lambda, pode tornar o código mais enxuto e direto, eliminando a necessidade de definições formais que seriam usadas apenas uma vez. Isso é especialmente útil quando se deseja evitar a criação de funções nomeadas para tarefas simples, como pequenas transformações de dados ou operações pontuais, garantindo uma abordagem mais concisa e legível no desenvolvimento.

Em Python, funções lambda não apresentam vantagens significativas de desempenho em relação às funções tradicionais definidas com `def`, pois ambas são tratadas internamente como objetos de primeira classe e executadas pelo interpretador da mesma forma. No entanto, lambda pode ter uma leve sobrecarga devido à sua limitação a uma única expressão e à impossibilidade de conter múltiplas instruções, tornando-as menos legíveis e mais difíceis de depurar. Além disso, ao serem usadas excessivamente em contextos complexos, como loops aninhados ou operações de alta carga, podem comprometer a clareza do código sem ganhos de eficiência, já que o Python não realiza otimizações específicas para lambda, diferentemente de linguagens compiladas que aplicam técnicas como `inlining`.



PARA SABER MAIS

As funções lambda em Python são úteis para expressões curtas e rápidas, mas sua utilização excessiva pode dificultar a depuração e manutenção do código. Para evitar erros comuns, como escopo inadequado de variáveis e uso indevido em expressões complexas, é recomendável testá-las em pequenos trechos de código antes de integrá-las a um programa maior.

Na área de probabilidade e estatística, Python oferece bibliotecas poderosas como NumPy, SciPy, pandas e statsmodels, que permitem cálculos estatísticos, modelagem de distribuições, testes de hipóteses e visualização de dados. Funções lambda são frequentemente usadas em conjunto com `map()`, `filter()` e `apply()` para manipulação eficiente de dados estatísticos, mas sua aplicação deve ser equilibrada com funções normais (`def`) para garantir clareza e reusabilidade do código.

Se deseja aprofundar seus conhecimentos sobre a correta utilização de funções lambda e programação estatística em Python, explore conceitos como funções de distribuição de probabilidade, regressão estatística e processamento vetorial com NumPy.

2 Uso das funções sort, map e list

A linguagem de programação Python oferece diversas funções embutidas para manipulação de listas e coleções de dados, sendo `sort`, `map` e `list` algumas das mais utilizadas. Cada uma delas tem uma aplicação específica e pode ser combinada com funções lambda para tornar o código mais conciso e eficiente.

Por exemplo, a função `sort()` é utilizada para ordenar listas in-place (ou seja, altera a lista original). Já `sorted()` retorna uma nova lista ordenada sem modificar a original. Observe, no exemplo 2, o código para analisar as diferenças. Lembrando que o argumento `reverse=True` permite realizar a ordenação de forma decrescente.

Exemplo 2

```
numeros = [5, 2, 9, 1, 5, 6]

# Ordenando a própria lista
numeros.sort()
print(numeros) # [1, 2, 5, 5, 6, 9]

# Ordenando sem modificar a lista original
nova_lista = sorted(numeros, reverse=True)
print(nova_lista) # [9, 6, 5, 5, 2, 1]
```

Podemos utilizar a opção `key` para definir critérios específicos, como ordenar uma lista de tuplas pelo segundo elemento, associando o recurso de função `lambda` para selecionar o elemento desejado da estrutura.

```
pessoas = [("Alice", 25), ("Bob", 30), ("Carol", 20)]
pessoas.sort(key=lambda x: x[1]) # Ordena pela idade
print(pessoas) # [('Carol', 20), ('Alice', 25), ('Bob', 30)]
```

Por sua vez, a função `map()` aplica uma função a cada item de um iterável, como listas e tuplas, retornando um iterador. Para obter uma lista, é necessário converter o resultado com `list()`, como o exemplo 3, que dobra os valores de uma lista.

Exemplo 3

```
numeros = [1, 2, 3, 4, 5]
dobrados = list(map(lambda x: x * 2, numeros))
print(dobrados) # [2, 4, 6, 8, 10]
```

Em um outro exemplo, o `map()` pode ser usado para formatar uma lista de strings, como apresentado a seguir, no exemplo 4.

Exemplo 4

```
nomes = ["alice", "bob", "carol"]
nomes_maiusculos = list(map(str.upper, nomes))
print(nomes_maiusculos) # ['ALICE', 'BOB', 'CAROL']
```

Por último, vamos fazer referência especificamente à função `list()`, usada para converter iteráveis (Iterator) em listas. Isso é útil ao trabalhar com iteradores, `map()`, `filter()`, `zip()`, entre outros.

O exemplo 5 é a conversão da saída da função `range()` para uma lista.

Exemplo 5

```
numeros = list(range(5))  
print(numeros) # [0, 1, 2, 3, 4]
```

Por sua vez, como já apresentado anteriormente, a função `list()` também converte iteradores em listas geradas pela função `map()`, como apresenta o exemplo 6 a seguir.

Exemplo 6

```
valores = map(lambda x: x**2, [1, 2, 3, 4])  
valores_lista = list(valores)  
print(valores_lista) # [1, 4, 9, 16]
```

Conhecendo as simplificações das funções `lambda` e de iteradores como o `map`, pode-se resolver problemas com mais facilidade e menos linhas de código. Vamos ao exemplo 7.

Exemplo 7

Suponha que você recebeu uma lista de temperaturas em graus Celsius e precisa convertê-las para Fahrenheit usando Python.

Primeiramente, vamos resolver este problema utilizando um código imperativo e tradicional, sem o uso de funções.

```
# Lista de temperaturas em Celsius  
celsius = [0, 10, 20, 30, 40, 100]  
  
# Criando uma lista vazia para armazenar os resultados  
fahrenheit = []
```

```
# Iterando sobre a lista e convertendo cada valor
for temp in celsius:
    f = (temp * 9/5) + 32 # Aplicando a fórmula de conversão
    fahrenheit.append(f) # Adicionando o resultado à nova
    lista

# Exibindo a lista de temperaturas em Fahrenheit
print(fahrenheit)
```

Criamos uma lista vazia chamada *fahrenheit* para armazenar os valores convertidos e, em seguida, percorremos a lista *celsius* usando um loop *for*, processando cada temperatura individualmente. Durante a iteração, aplicamos a fórmula de conversão:

$$F = \left(C \times \frac{9}{5}\right) + 32$$

Depois, armazenamos o resultado na variável *f*. Esse valor convertido é então adicionado à lista *fahrenheit* utilizando o método *.append()*. Após a conclusão do loop, imprimimos a lista resultante contendo todas as temperaturas convertidas de Celsius para Fahrenheit.

Contudo, vejamos como podemos refatorar esse código para o uso de funções tradicionais, como mostra o exemplo 8.

Exemplo 8

```
# Função para converter Celsius para Fahrenheit
def celsius_para_fahrenheit(c):
    return (c * 9/5) + 32

# Lista de temperaturas em Celsius
celsius = [0, 10, 20, 30, 40, 100]

# Criando uma lista vazia para armazenar as temperaturas
# convertidas
fahrenheit = []

# Percorrendo a lista celsius elemento por elemento
```

```

for temp in celsius:
    # Convertendo a temperatura atual de Celsius para
    Fahrenheit
    temp_fahrenheit = celsius_para_fahrenheit(temp)

    # Adicionando a temperatura convertida à lista fahrenheit
    fahrenheit.append(temp_fahrenheit)

# Exibindo a lista final de temperaturas convertidas
print(fahrenheit)

```

Nesse caso, a fórmula para a transformação de escala de temperatura foi encapsulada na função `celsius_para_fahrenheit()`, seguindo uma melhor organização e encapsulamento. No entanto, podemos otimizar ainda mais essa solução utilizando funções `map()` e `lambda()`, conforme exemplo 9.

Exemplo 9

```

# Lista de temperaturas em Celsius
celsius = [0, 10, 20, 30, 40, 100]

# Aplicando a conversão com map() e lambda
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))

# Exibindo o resultado
print(fahrenheit)

```

Criamos uma lista contendo temperaturas em Celsius e utilizamos a função `map()`, que permite aplicar uma transformação a cada elemento dessa lista de forma eficiente. Dentro do `map()`, empregamos uma função `lambda` para aplicar diretamente a fórmula de conversão de Celsius para Fahrenheit, tornando o código mais conciso. Como `map()` retorna um iterador, utilizamos `list()` para convertê-lo em uma lista, garantindo que os valores possam ser manipulados e exibidos facilmente. Por fim, imprimimos a nova lista contendo as temperaturas já convertidas para Fahrenheit.

Considerações finais

As funções lambda são um recurso poderoso em Python para simplificar operações curtas e tornar o código mais enxuto e expressivo, especialmente quando usadas em conjunto com funções como `sort`, `map` e `list`. Embora sua utilização facilite a escrita de código conciso, é essencial equilibrar seu uso com funções tradicionais (`def`) para garantir legibilidade e manutenção. Em contextos mais complexos, o uso excessivo de funções lambda pode comprometer a clareza do código, tornando a depuração mais difícil. Portanto, compreender quando e como aplicá-las corretamente é fundamental para um desenvolvimento eficiente e organizado em Python.

Referências

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.

Tratamento de exceções

O desenvolvimento de programas, mesmo quando são utilizados algoritmos eficientes e bem estruturados, pode enfrentar situações que fogem ao controle da pessoa desenvolvedora. Erros decorrentes de entradas de dados inadequadas ou indisponibilidade de serviços são comuns durante a execução de programas, podendo causar interrupções e impactar negativamente a experiência do usuário. Para lidar com esses desafios, as linguagens de programação oferecem mecanismos de tratamento de exceções, que permitem identificar essas inconsistências e aplicar soluções que garantam a estabilidade do software (Guilhon *et al.*, 2022; Lambert, 2022).

Nesse contexto, o primeiro subcapítulo aborda os conceitos e aplicações do tratamento de exceções na linguagem Python, explorando diferentes cenários de uso. O segundo subcapítulo discute como a pessoa desenvolvedora pode definir seus próprios critérios de tratamento de erros, disparando as exceções de maneira personalizada.

1 Conceito e aplicações

Quando nos referimos ao tratamento de exceção em linguagens de programação, é importante ressaltar que exceções e erros de sintaxe são situações diferentes. O erro de sintaxe está relacionado à escrita do código-fonte de maneira incompatível com o interpretador ou compilador da linguagem de programação alvo. Já uma exceção acontece mesmo com o código-fonte escrito corretamente, o que depende de situações que ocorrem em tempo de execução e que podem ser tratadas amigavelmente de maneira a não interromper o fluxo de execução de um programa. A sintaxe básica para o tratamento de exceção é:

```
try:
    # Código que pode gerar uma exceção
except ExceptionType:
    # Código a ser executado se a exceção ocorrer
```

Em que a palavra reservada *try* circunda o bloco de instruções que deve ser observado quanto às exceções que podem ser geradas, e *except* é o tipo de exceção que deve ser capturado no trecho circundado pelo bloco *try*. Vamos ao exemplo 1.

Exemplo 1

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("Erro: divisão por zero não permitida!")
```

O exemplo 1 provoca um erro a partir de uma operação de divisão por zero não permitida. Essa instrução, sem o devido tratamento, geraria uma interrupção na aplicação informando uma exceção de divisão por zero (*ZeroDivisionError*). No entanto, com a utilização da estrutura de tratamento de exceção, o programa, em vez de gerar o erro e encerrar, exibe a mensagem "Erro: divisão por zero não permitida!", contemplada no bloco *except*.

Além da captura de uma única exceção, podem ser capturadas mais exceções realizando o tratamento em específico por cada bloco `except`, como no exemplo 2 a seguir, no qual há a captura de erros do tipo `ZeroDivisionError` ou `ValueError` (por exemplo, se o usuário insere um caractere não identificado por um número) em blocos de instrução separados.

Exemplo 2

```
try:
    num = int(input("Digite um número: "))
    resultado = 10 / num
except ValueError:
    print("Erro: você deve digitar um número inteiro válido.")
except ZeroDivisionError:
    print("Erro: divisão por zero não é permitida.")
else:
    print(f"Resultado: {resultado}")
```

Além do uso dos blocos `try` e `except`, os blocos `else` e `finally` também podem ser utilizados nas seguintes situações: o bloco `else` é executado caso nenhuma exceção seja capturada no bloco `try`, e o bloco `finally` sempre é executado, independentemente de ser capturada uma exceção ou não, com mostra o exemplo 3.

Exemplo 3

```
try:
    arquivo = open("dados.txt", "r")
    conteudo = arquivo.read()
except FileNotFoundError:
    print("Arquivo não encontrado.")
else:
    print("Arquivo lido com sucesso!")
finally:
    print("Encerrando o processo.")
```

O código tenta abrir o arquivo `"dados.txt"` em modo de leitura (`"r"`) dentro do bloco `try`. Se o arquivo não for encontrado, a exceção

`FileNotFoundError` será capturada pelo bloco `except`, exibindo a mensagem “Arquivo não encontrado”. Caso o arquivo seja encontrado e lido com sucesso, a mensagem “Arquivo lido com sucesso!” será exibida no bloco `else`. Independentemente de ocorrer erro ou não, o bloco `finally` sempre será executado, imprimindo “Encerrando o processo”, garantindo que qualquer ação final necessária seja realizada. O quadro 1 mostra os diferentes tipos de exceção que podem ser capturadas na linguagem Python.

Quadro 1 – Tipos de exceções mais comuns em Python

TIPO DE EXCEÇÃO	DESCRIÇÃO
<code>Exception</code>	Classe base para todas as exceções em Python.
<code>ArithmeticError</code>	Classe base para erros matemáticos.
<code>ZeroDivisionError</code>	Erro ao tentar dividir um número por zero.
<code>OverflowError</code>	Erro quando uma operação aritmética ultrapassa o limite representável.
<code>ValueError</code>	Erro quando um argumento tem um valor inadequado para uma operação.
<code>TypeError</code>	Erro quando uma operação ou função é aplicada a um objeto de tipo inadequado.
<code>IndexError</code>	Erro quando um índice de uma sequência está fora do intervalo válido.
<code>KeyError</code>	Erro quando uma chave não é encontrada em um dicionário.
<code>FileNotFoundError</code>	Erro ao tentar abrir um arquivo que não existe.
<code>IOError</code>	Erro ao lidar com operações de entrada e saída.
<code>ImportError</code>	Erro ao tentar importar um módulo que não pode ser encontrado.
<code>ModuleNotFoundError</code>	Erro quando um módulo específico não pode ser encontrado.
<code>AttributeError</code>	Erro ao tentar acessar um atributo inexistente de um objeto.
<code>NameError</code>	Erro quando uma variável ou função não foi definida antes do uso.
<code>IndentationError</code>	Erro quando a indentação do código está incorreta.
<code>SyntaxError</code>	Erro de sintaxe no código fonte do Python.

TIPO DE EXCEÇÃO	DESCRIÇÃO
RuntimeError	Erro genérico que ocorre durante a execução do programa.
MemoryError	Erro quando o sistema fica sem memória suficiente para uma operação.
AssertionError	Erro quando uma asserção assert falha.
KeyboardInterrupt	Erro gerado quando o usuário interrompe a execução do programa (Ctrl+C).

2 Invocando exceções manualmente

Além das exceções geradas nativamente em tempo de execução, as mesmas exceções podem ser geradas manualmente em situações não mapeadas através da palavra reservada `raise`, como apresentado no exemplo 4 a seguir.

Exemplo 4

```
def verificar_idade(idade):
    if idade < 18:
        raise ValueError("Idade deve ser maior ou igual a 18.")
    return "Acesso permitido."

try:
    print(verificar_idade(16))
except ValueError as e:
    print(f"Erro: {e}")
```

O código define a função `verificar_idade(idade)`, que verifica se a idade fornecida é menor que 18; caso seja, a função dispara uma exceção `ValueError` com a mensagem "Idade deve ser maior ou igual a 18". No bloco `try`, a função é chamada com o valor 16, o que aciona a exceção. O bloco `except` captura essa exceção e imprime a mensagem de erro formatada com o texto "Erro: Idade deve ser maior ou igual a 18", evitando que o programa seja interrompido abruptamente.

Para um uso eficaz de exceções em Python, é fundamental evitar capturas genéricas com `except Exception`, pois isso pode mascarar erros inesperados e dificultar a depuração, tornando o código menos previsível. Em vez disso, é recomendável capturar exceções específicas para lidar com cada situação de maneira apropriada. Além disso, fornecer mensagens de erro claras e informativas é essencial para facilitar a identificação e correção do problema, indicando exatamente o que aconteceu e, se possível, sugerindo uma solução.



PARA SABER MAIS

Em Python é possível personalizar a geração de exceções! Para elaborar classes personalizadas de exceção em Python, é necessário criar uma classe que herde da classe base `Exception`, permitindo definir erros específicos para um contexto particular da aplicação. Isso é útil quando os erros padrão não representam adequadamente uma situação ou quando se deseja adicionar lógica personalizada ao tratamento de exceções. Dentro da classe, pode-se sobrescrever o método de inicialização para aceitar mensagens de erro personalizadas ou outros parâmetros relevantes. Ao utilizá-la, a exceção pode ser lançada com uma mensagem clara e capturada de forma específica, melhorando a legibilidade do código, facilitando a depuração e permitindo um tratamento mais preciso dos erros.

Outra boa prática é utilizar logs em vez de `print()`, pois a biblioteca logging permite armazenar erros com diferentes níveis de gravidade (DEBUG, INFO, WARNING, ERROR, CRITICAL), além de possibilitar o envio dessas informações para arquivos ou sistemas externos, garantindo um melhor monitoramento e rastreamento de falhas no código. O código a seguir (exemplo 5) apresenta um exemplo do uso de logs em Python.

Exemplo 5

```
import logging

logging.basicConfig(level=logging.ERROR)

try:

    x = 1 / 0
except ZeroDivisionError as e:
    logging.error(f"Erro detectado: {e}")
```

O código utiliza a biblioteca logging para registrar erros em vez de simplesmente imprimir mensagens na tela. Primeiro, o módulo logging é importado, e a configuração logging.basicConfig(level=logging.ERROR) define que apenas mensagens de nível ERROR ou superior serão registradas. No bloco try, o programa tenta dividir 1 por 0, o que gera uma exceção ZeroDivisionError. O bloco except captura essa exceção e usa logging.error(f"Erro detectado: {e}") para registrar a mensagem de erro, em que {e} contém a descrição da exceção ("division by zero"). Isso permite que o erro seja tratado de maneira estruturada e registrado em logs, facilitando a depuração sem interromper a execução do programa.



IMPORTANTE

Os logs e a observabilidade são fundamentais para a manutenção e o monitoramento de servidores, permitindo detectar falhas, analisar o desempenho e garantir a segurança das aplicações. Enquanto os logs registram eventos detalhados sobre o funcionamento do sistema, incluindo erros, requisições e comportamento dos serviços, a observabilidade vai além, fornecendo uma visão holística do ambiente por meio de métricas, rastreamento distribuído e alertas proativos. O uso adequado de ferramentas de logging e monitoramento, como ELK Stack, Prometheus e Grafana, possibilita identificar gargalos, prever problemas e responder rapidamente a incidentes, garantindo maior estabilidade, eficiência e confiabilidade no ambiente de produção.

Considerações finais

É fundamental reforçar a importância de lidar com erros de forma eficiente para manter a estabilidade e previsibilidade das aplicações. Usar try-except corretamente, criar exceções personalizadas e seguir boas práticas, como evitar capturas genéricas e fornecer mensagens claras, tornam a depuração mais fácil e melhoram a experiência do usuário. Além disso, logs estruturados ajudam a monitorar o comportamento do sistema em tempo real, facilitando a detecção de problemas. Com essas estratégias, o software se torna mais robusto, confiável e preparado para enfrentar falhas de forma controlada.

Referências

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.

Arquivos

Uma das importantes práticas de programação para profissionais e estudantes que desenvolvem serviços e aplicações é a devida manipulação de arquivos em seus diferentes tipos e propósitos de armazenamento. Mesmo sendo um formato pioneiro no transporte e armazenamento em memória secundária – termo utilizado para se referir a memórias não voláteis, como discos rígidos e memórias de estado sólido –, os arquivos são amplamente utilizados para finalidades como intercâmbio de dados entre aplicações (JSON, XML, CSV etc.) e configurações de serviços (YAML, INI, ENV etc.), sem esquecer dos arquivos com dados fora de um padrão específico, conhecidos como dados RAW (Guilhon *et al.*, 2022; Lambert, 2022).

Além da família de arquivos texto, existem também os arquivos binários com formatos conhecidos, tais como mídias nos seus diferentes tipos (JPEG, PNG, MP3, MP4, AVI etc.) e arquivos com dados compactados ou executáveis (ZIP, ARJ, EXE, BIN etc.), cada um com suas particularidades na manipulação.

Este capítulo tem por propósito apresentar as práticas de programação voltada para a manipulação dos diferentes tipos de arquivos, destacando suas particularidades e finalidades de uso. O primeiro subcapítulo apresenta como arquivos textos são importantes no intercâmbio de dados, incluindo suas técnicas de manipulação. Depois, serão mostradas as particularidades na manipulação de arquivos binários.

1 Arquivos texto

Na programação em linguagem Python, a manipulação de textos é nativa, contendo funções e objetos que facilitam sua manipulação. Existem algumas particularidades quando se deseja trabalhar com arquivos textos que precisam ser levados em conta, tal como a codificação, de forma que cada solução de software adote um conjunto de caracteres dentro de um padrão, o qual está correlacionado com o espaço que esse caractere ocupa em memória secundária, além de questões regionais em sua adoção. O quadro 1 apresenta as principais codificações de arquivos existentes. Entre as codificações mais populares, encontra-se a UTF-8 (padrão utilizado em idiomas do Ocidente) ou UTF-16 (padrão de 2 ou 4 bytes utilizado em sistemas asiáticos).

Quadro 1 – Codificações de arquivos

CODIFICAÇÃO	CARACTERÍSTICAS PRINCIPAIS	USO COMUM
UTF-8	Codificação universal, compatível com ASCII, suporta todos os caracteres Unicode	Padrão na web, Python, JSON, arquivos modernos
UTF-16	Representa caracteres Unicode com 2 ou 4 bytes, pode conter byte order mark (BOM)	Windows, arquivos XML, sistemas asiáticos
UTF-32	Usa 4 bytes fixos por caractere, muito simples, mas ocupa mais espaço	Casos específicos que exigem performance previsível
ASCII	Codifica apenas 128 caracteres (inglês básico), leve e compatível com sistemas antigos	Arquivos simples, sistemas legados

cont.

CODIFICAÇÃO	CARACTERÍSTICAS PRINCIPAIS	USO COMUM
ISO-8859-1	Também conhecida como Latin-1, inclui acentos usados em português e outras línguas latinas	Sistemas legados e antigos arquivos em português
Windows-1252	Extensão do ISO-8859-1 com símbolos extras	Windows antigos, arquivos Word etc.



PARA SABER MAIS

Ao lidar com arquivos de texto em Python (e em praticamente qualquer linguagem de programação), é comum nos depararmos com termos como UTF-8, UTF-16 ou Unicode. Mas, afinal, o que significa Unicode?

Unicode é um padrão universal de codificação de caracteres, criado para resolver um problema antigo: a falta de um sistema único e consistente para representar todos os símbolos usados nas linguagens humanas – incluindo letras, números, acentos, emojis, alfabetos não latinos e símbolos técnicos.

Antes do Unicode, cada sistema ou país criava seu próprio conjunto de códigos, como o ASCII, ISO-8859-1 ou Windows-1252, o que gerava confusões e arquivos incompatíveis entre si. O Unicode surgiu para unificar tudo isso, atribuindo a cada caractere um código numérico único, chamado de code point.

Depois de reconhecer os diferentes tipos de codificação existentes, a próxima etapa é entender o processo primordial de arquivos, isto é, sua abertura e fechamento. A abertura consiste em receber um objeto no qual se faz a gestão de manipulação de arquivos: localiza-se o arquivo, disponibiliza-se para manipulação e depois realiza-se o fechamento.

O Python possui duas estruturas nativas na manipulação de arquivos em que a diferença está no processo de fechamento do arquivo. O primeiro exemplo está relacionado ao uso da função `open()`, a qual necessariamente abre um arquivo, imprime o conteúdo e faz o fechamento, como segue.

Exemplo 1

```
arquivo = open("exemplo.txt", "r") # abre para leitura
conteudo = arquivo.read()
print(conteudo)
arquivo.close()
```

Algumas considerações são importantes no código-fonte apresentado no exemplo 1. Primeiramente, no comando `open()`, é passado como argumento o caminho do arquivo (`exemplo.txt`) para a uma operação de leitura (destacado com `"r"`). Na abertura do arquivo, podem ser escolhidos diferentes tipos de operação, permitindo a escrita, leitura ou ambas as operações. O quadro 2 mostra os possíveis modos de abertura do arquivo.

Quadro 2 – Modos de abertura

MODO	SIGNIFICADO	DESCRIÇÃO
'r'	Read	Abre para leitura. Gera erro se o arquivo não existir.
'w'	Write	Abre para escrita. Cria o arquivo se não existir, sobrescreve se existir.
'a'	Append	Abre para adição no final do arquivo. Cria o arquivo se não existir.
'x'	Create	Cria um arquivo. Gera erro se o arquivo já existir.
'b'	Binary	Modo binário (adicionado a outros modos, como 'rb' ou 'wb').
't'	Text (padrão)	Modo texto (padrão, pode ser omitido). Usado com 'r', 'w' etc.
'+'	Read and write	Abre para leitura e escrita ao mesmo tempo. Usado com 'r+', 'w+' etc.



IMPORTANTE

Ao lidar com Python, os modos de abertura de arquivos podem ser combinados para oferecer mais controle sobre como o arquivo será manipulado. Por exemplo, ao utilizar `'rb'`, o arquivo é aberto para leitura em modo binário – ideal para imagens, áudios e vídeos, que não são lidos como

texto. Já o modo 'w+' permite escrever e ler no mesmo arquivo, mas apaga todo o conteúdo anterior caso o arquivo já exista. O modo 'a+' é útil quando se deseja adicionar novos dados ao final de um arquivo existente, sem apagar o conteúdo existente, ao mesmo tempo que se mantém a possibilidade de leitura. Outro exemplo é 'x+', que permite criar um arquivo para leitura e escrita, gerando erro se o arquivo já existir, o que o torna útil em situações em que é importante evitar sobrescritas acidentais. Esses modos combinados dão mais flexibilidade ao programador, mas exigem atenção para evitar perda de dados ou comportamentos inesperados.

O próximo exemplo é realizar a abertura do arquivo, porém, utilizando a estrutura `with open`, a qual tem como particularidade a garantia do fechamento do arquivo. A estrutura estabelece que toda a operação com o arquivo deve ser realizada dentro de um bloco de execução. Quando o programa encerra a execução do bloco, automaticamente o arquivo é fechado. Vamos compreender com o exemplo 2.

Exemplo 2

```
with open("exemplo.txt", "r") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)
# Arquivo é fechado automaticamente aqui
```

Perceba que toda a manipulação do arquivo é realizada dentro do bloco estabelecido. Quando encerrada a execução do bloco, o arquivo é encerrado e o objeto *arquivo* é desalocado da memória da aplicação, evitando esquecimentos no encerramento de arquivos.

Também existem outras maneiras de realizar a leitura do arquivo utilizando comandos como o `readline()`, que faz a leitura completa de uma linha por execução, e o comando `readlines()`, que faz a leitura de todo o arquivo organizado em listas de linhas, como demonstra o exemplo 3.

Exemplo 3

```
with open("exemplo.txt", "r") as arquivo:
    linha1 = arquivo.readline()
    linhas = arquivo.readlines()
    print(linha1)
    print(linhas)
```

Para a escrita em arquivos, são utilizados comandos como o `write()`, em que se escreve um texto qualquer no arquivo, e o comando `writelines()`, em que se pode escrever um conjunto de linhas organizado por listas. Um cuidado importante na escrita é a escolha do modo de abertura: "w" indica que o arquivo será sobrescrito, e "a" indica que o conteúdo será gravado ao final do arquivo.

Exemplo 3

```
# Sobrescreve o conteúdo
with open("saida.txt", "w") as arquivo:
    arquivo.write("Primeira linha.\n")
    arquivo.write("Segunda linha.\n")

# Adiciona ao final do arquivo
with open("saida.txt", "a") as arquivo:
    arquivo.write("Linha adicionada.\n")
```

No exemplo 3, identificamos que, na primeira manipulação do arquivo `saida.txt`, o segundo comando `write()` sobrescreverá a escrita anterior, enquanto a segunda manipulação está com o modo de abertura "a" e sempre adicionará uma nova linha ao final.

Outros dois pontos importantes na manipulação de arquivos estão na escolha da codificação e no tratamento de erros. Antes de realizar a abertura de um arquivo, pode ser especificado o argumento `encoding` para garantir que tal manipulação respeita um tipo de codificação. Veja um exemplo em que é informado que o padrão da codificação é UTF-8. Essa configuração evita problemas com acentos e caracteres especiais.

Exemplo 4

```
with open("exemplo.txt", "r", encoding="utf-8") as arquivo:
    print(arquivo.read())
```

Para evitar encerramentos desnecessários na aplicação, as funções e os objetos de manipulação de arquivos geram exceções para algumas situações; por exemplo, quando o arquivo não é encontrado ou quando a aplicação não possui autorização para acesso. Para capturar e tratar a exceção, o código-fonte deve ser circundado com o bloco try, e cada exceção pode ter sua resposta programada de maneira personalizada utilizando bloco except, como mostra o exemplo 5.

Exemplo 5

```
try:
    with open("inexistente.txt", "r") as arquivo:
        conteudo = arquivo.read()
except FileNotFoundError:
    print("Arquivo não encontrado.")
```

No exemplo 5, o programa está preparado para uma situação caso o arquivo não seja encontrado, fornecendo uma resposta amigável. O quadro 3 apresenta as principais exceções que podem ser capturadas no processo de manipulação de arquivos.

Quadro 3 – Exceções para arquivos em Python

EXCEÇÃO	QUANDO OCORRE	EXEMPLO DE SITUAÇÃO
FileNotFoundError	Quando o arquivo não é encontrado no modo de leitura ('r' ou 'r+')	Tentar abrir um arquivo que não existe
PermissionError	Quando o programa não tem permissão para acessar ou modificar o arquivo	Tentar abrir um arquivo de sistema sem privilégios
IsADirectoryError	Quando se tenta abrir uma pasta como se fosse um arquivo	Usar open("caminho/") em vez de open("arquivo.txt")

cont.

EXCEÇÃO	QUANDO OCORRE	EXEMPLO DE SITUAÇÃO
NotADirectoryError	Quando se tenta acessar um caminho inválido, tratando um arquivo como pasta	<code>open("arquivo.txt/outro.txt")</code>
IOError ou OSError	Erros genéricos de entrada/saída (I/O), como falhas de leitura, disco cheio etc.	Problema físico ou lógico ao acessar o arquivo
UnicodeDecodeError	Quando a leitura falha por usar a codificação errada	Tentar ler um arquivo em UTF-16 usando <code>encoding='utf-8'</code>
UnicodeEncodeError	Quando ocorre erro ao tentar salvar texto com caracteres incompatíveis	Tentar salvar acentos em um arquivo com <code>encoding='ascii'</code>
ValueError	Quando o modo de abertura do arquivo é inválido ou está mal formatado	Usar <code>open("arquivo.txt", "rw")</code> , que não é um modo válido

2 Arquivos binário

Diferentemente da manipulação de dados em texto, os arquivos binários, como o nome sugere, possuem uma manipulação refinada a nível de cada byte, levando em consideração as diferentes codificações particulares, principalmente voltadas para arquivos executáveis, compactados ou de mídia.

Os arquivos, quando sinalizados para serem tratados com formatos binários, não passam por decodificação automática tal como os arquivos textos UTF-8 ou ASCII. Os arquivos, dessa maneira, são disponibilizados em formato de um conjunto de bytes com formato bytearray. O exemplo 6 apresenta o uso do comando `with open` na abertura de um arquivo binário.

Exemplo 6

```
with open("audio.mp3", "rb") as arquivo:
    dados = arquivo.read(1024) # lê os primeiros 1024 bytes
```

Nesse caso, é importante observar que a sinalização para a manipulação do arquivo em formato binário é a inclusão da letra “b” no modo de abertura; nesse caso, “rb”, em que o arquivo será aberto em formato binário para leitura *sem a decodificação automática*. O comando `read()`, por sua vez, armazena na variável `dados` os primeiros 1024 bytes do arquivo `audio.mp3`.

Com o uso de laços, é possível determinar os blocos para processamento de todo um arquivo binário. No exemplo a seguir, um arquivo de vídeo MP4 pode ser processado em divisões de 4KB utilizando o comando `read()` em conjunto com o laço `while` até que a leitura alcance o final do arquivo (chamado de EOF – end of file). O operador `:=`, conhecido como “operador walrus” (ou “operador morsa”), foi introduzido no Python 3.8 e permite a atribuição de valores a variáveis dentro de expressões. Ele é especialmente útil para evitar repetições, simplificar códigos e torná-los mais concisos, como em estruturas condicionais ou loops. Por exemplo, em vez de escrever um bloco de código em que uma função é chamada duas vezes (uma para verificar o valor retornado e outra para usá-lo) o operador `:=` permite que o resultado seja atribuído a uma variável e usado simultaneamente na expressão. Seu uso comum é dentro de um `while`, como em `while (linha := arquivo.readline())`; que lê uma linha do arquivo e avalia se ainda há conteúdo; tudo em uma única instrução. Essa funcionalidade favorece a legibilidade e eficiência do código, desde que usada com parcimônia para não prejudicar a clareza.

Exemplo 7

```
with open("video.mp4", "rb") as arquivo:
    while chunk := arquivo.read(4096):
        # processa bloco de 4KB
        pass
```

Um outro exemplo de operação com arquivos binários é realizar sua cópia combinando os comandos `write()` e `read()` de maneira indentada, como segue.

Exemplo 8

```
with open("original.pdf", "rb") as origem:
    with open("copia.pdf", "wb") as destino:
        destino.write(origem.read())
```



IMPORTANTE

Os arquivos binários são fundamentais para o funcionamento de diversas aplicações no mundo digital, pois permitem o armazenamento e a manipulação eficiente de dados complexos que não podem ser representados apenas como texto, imagens, vídeos, áudios, documentos, planilhas e programas executáveis. Diferentemente dos arquivos de texto, que armazenam caracteres legíveis, os arquivos binários contêm sequências de bytes que representam diretamente estruturas internas, compactadas ou codificadas, exigindo programas específicos para sua interpretação. Essa capacidade de representar dados ricos em informação, com formatos compactos e performáticos, torna os arquivos binários indispensáveis em áreas como multimídia, jogos, sistemas embarcados, bancos de dados, engenharia e até mesmo no armazenamento de modelos de inteligência artificial. Em Python, trabalhar com arquivos binários exige atenção especial ao modo de abertura ('rb', 'wb') e à manipulação de dados no formato de bytes, garantindo precisão e fidelidade na leitura e gravação dessas informações.

Considerações finais

A manipulação de arquivos é uma competência essencial na programação, pois permite que aplicações armazenem, leiam e troquem informações de forma eficiente e persistente. Ao longo deste capítulo, foram apresentadas as principais práticas relacionadas à manipulação de arquivos textos e binários na linguagem Python, destacando particularidades, modos de abertura, cuidados com codificação, tratamento de exceções e formas de leitura e escrita. Os arquivos textos, amplamente utilizados para configurações e intercâmbio de dados entre sistemas, exigem

atenção à codificação de caracteres, enquanto os arquivos binários são fundamentais para lidar com dados complexos, como imagens, áudios e documentos. Compreender esses conceitos permite que o programador atue de forma mais segura, organizada e eficiente no desenvolvimento de soluções que dependem de leitura e gravação em memória secundária, respeitando os formatos e padrões esperados em cada tipo de aplicação.

Referências

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.

Gerenciamento de módulos

Linguagens de programação populares tal como o Python são mantidas por comunidades de pessoas e organizações interessadas em seu crescimento e disseminação nas diferentes soluções da tecnologia da informação. Entender o funcionamento da comunidade é importante para compreender como os módulos são desenvolvidos e disseminados por ela. Este capítulo, em essencial, trata das informações pertinentes ao gerenciamento dos inúmeros módulos existente na comunidade de desenvolvedores Python.

O primeiro subcapítulo trata de como é formada a comunidade de desenvolvedores do Python, justificando sua grande disseminação nas diversas aplicações voltadas à tecnologia da informação. Depois, será apresentado como utilizar o gerenciador de módulos pip, imprescindível para a gestão de ferramentas em Python, assim como entender a importância e o funcionamento dos ambientes virtuais. Ao final, entenderemos como criamos nossos próprios módulos para serem reaproveitados nas diferentes aplicações.

1 A comunidade de desenvolvimento Python

Desde que Guido van Rossum criou a linguagem Python, em 1989, ela tem evoluído significativamente, não se limitando ao aspecto técnico. Python cresceu como reflexo de uma comunidade colaborativa que ajudou a moldar a cultura moderna do desenvolvimento de software. Desde o início, a proposta foi clara: criar uma linguagem *simples, legível e acessível*. E foi justamente essa combinação que conquistou tanto quem está começando quanto programadores experientes.

Com o tempo, a comunidade de desenvolvedores Python se firmou como uma das mais inclusivas da programação. Eventos como a PyCon acontecem em vários países, e iniciativas como a Python Software Foundation (PSF) são exemplos de como a linguagem se apoia em valores como diversidade, educação e inovação (Pycon, 2025; Python, 2025). A filosofia expressa no Zen of Python, especialmente o princípio “There should be one – and preferably only one – obvious way to do it” (“Deve haver uma – e preferencialmente apenas uma – maneira óbvia de fazer isso”), reforça uma cultura que preza pela clareza, por boas práticas e pelo trabalho em conjunto.

Um repositório importante para a comunidade de desenvolvedores é o Python Package Index (PyPI), o repositório oficial de pacotes de software para a linguagem de programação Python, desempenhando um papel fundamental no ecossistema da linguagem ao facilitar o compartilhamento e a reutilização de código. Por meio dele, desenvolvedores do mundo todo podem publicar bibliotecas, frameworks e utilitários que resolvem desde tarefas simples até problemas altamente complexos. Ao centralizar esses recursos em um único local, o PyPI promove a colaboração e o avanço da comunidade Python e acelera o desenvolvimento de projetos ao oferecer soluções prontas e bem documentadas. Dessa forma, o PyPI se consolida como uma ferramenta indispensável para quem deseja programar de forma eficiente, escalável e moderna (PyPI, 2015).

No Brasil, o uso de Python ganhou força a partir dos anos 2000, impulsionado por iniciativas da comunidade e por seu uso crescente em universidades e projetos de código aberto. A criação da Python Brasil (conferência nacional que reúne entusiastas, estudantes e profissionais) foi um marco importante para consolidar o ecossistema no país (Python Brasil, 2015). Além disso, grupos locais como os PUGs (Python User Groups) espalhados por diversas cidades ajudaram a democratizar o conhecimento, promovendo encontros, workshops e maratonas de programação. Python se popularizou entre cientistas de dados, desenvolvedores web e pesquisadores, tornando-se uma linguagem estratégica em setores como educação, governo, fintechs e tecnologia ambiental. Esse movimento comunitário, unido à versatilidade da linguagem, fez com que o Brasil se destacasse como uma das comunidades Python mais ativas do mundo.



PARA SABER MAIS

Python é uma linguagem conhecida por sua simplicidade e versatilidade, muito graças ao seu vasto ecossistema de frameworks, que ajudam a acelerar o desenvolvimento e aplicar boas práticas desde o início. Entre os mais populares está o Django, um framework completo para aplicações web, que já vem com autenticação, painel administrativo, ORM (object-relational mapping ou mapeamento objeto-relacional) e muito mais. O Flask, por sua vez, é mais leve e flexível, ideal para projetos menores ou quando se busca maior controle sobre os componentes utilizados. O FastAPI vem ganhando destaque como uma opção moderna e assíncrona para a construção de APIs rápidas e bem documentadas. Para quem trabalha com dados, o Pandas e o NumPy são ferramentas essenciais para manipulação e cálculos numéricos. Já no universo da inteligência artificial, frameworks como TensorFlow e PyTorch lideram o cenário, sendo o primeiro mais comum em ambientes de produção, e o segundo preferido em pesquisas por sua flexibilidade. E para quem deseja automatizar a extração de dados da web, o Scrapy é uma excelente escolha. Cada framework atende a um tipo de necessidade, e conhecer essas ferramentas é o primeiro passo para ampliar as possibilidades com Python e desenvolver projetos mais eficientes e profissionais.

2 O gerenciamento de módulos com pip

O ecossistema do Python é amplamente conhecido por sua riqueza de bibliotecas e pacotes que estendem as funcionalidades da linguagem para diferentes finalidades: desenvolvimento web, análise de dados, inteligência artificial, automação, entre muitas outras. Para lidar com essa diversidade, o Python utiliza o pip (acrônimo para `pip installs packages`) como seu principal gerenciador de pacotes. O pip é uma ferramenta que permite instalar, atualizar e remover pacotes Python com facilidade. Ele é incluído por padrão nas versões mais recentes do Python (a partir da 3.4), mas pode ser atualizado para a versão mais recente com o seguinte comando:

```
python -m pip install --upgrade pip
```

Para verificar a versão instalada, basta utilizar o comando pip com a opção “version”, como segue:

```
pip --version
```

A instalação de pacotes com o pip é realizada utilizando o comando `pip install`, adicionando também o nome do pacote que precisa ser instalado. Por exemplo, para instalar a biblioteca `requests` na versão mais recente, deve ser utilizado o comando:

```
pip install requests
```

Também é possível especificar a versão desejada para o pacote, através do comando:

```
pip install requests==2.31.0
```

Existem outras opções para a instalação de pacotes a partir de pastas locais, URLs ou arquivos “.whl” (wheel), conforme os exemplos a seguir, respectivamente.

Exemplo 1

```
pip install ./meu_pacote/  
pip install https://exemplo.com/arquivo.tar.gz  
pip install pacote_exemplo-1.0.0-py3-none-any.whl
```

Para verificar os pacotes instalados no ambiente atual, deve ser utilizado o comando pip com a opção list:

```
pip list
```

É possível obter detalhes de um pacote específico, como versão, dependências e localização, pela opção show:

```
pip show nome-do-pacote
```

Também é possível verificar pacotes desatualizados com a opção outdated:

```
pip list --outdated
```

Pacotes podem ser atualizados com a opção upgrade, adicionando ao final o nome do pacote, como apresentado a seguir.

```
pip install --upgrade nome-do-pacote
```

Para remover um pacote, pode ser usado o comando uninstall, adicionando ao final o nome do pacote, como mostrado na sequência

```
pip uninstall nome-do-pacote
```

Outro aspecto importante é a geração de um arquivo com as dependências de um ambiente para garantir que a execução de um aplicativo ou serviço seja realizada nas mesmas condições. Para isso, é comum utilizar um arquivo chamado requirements.txt, o qual pode ser gerado pela opção freeze, apontando a saída do arquivo para ser gravado em requirements.txt, como segue.

```
pip freeze > requirements.txt
```

Para instalar os pacotes de maneira a reproduzir o ambiente com todas as dependências, basta usar o comando `install` com a opção `-r` e o nome do arquivo com as dependências (na maioria dos casos, o `requirements.txt`).

```
pip install -r requirements.txt
```

Dominar o uso do `pip` é essencial para qualquer profissional que desenvolva com Python. Além de facilitar a instalação e manutenção de bibliotecas, ele promove organização, reprodutibilidade e escalabilidade nos projetos.

3 Ambiente virtual

Uma das boas práticas mais importantes no desenvolvimento em Python é a utilização de ambientes virtuais, que isolam as dependências de cada projeto, evitando conflitos entre versões de pacotes.

Para criar um ambiente virtual, basta utilizar o módulo `venv` do Python, indicando nome do ambiente. Serão criados uma pasta referente ao nome do ambiente e os arquivos internos para instanciar o ambiente em seu sistema operacional, como segue.

```
python -m venv nome_do_ambiente
```

Para ativar o ambiente virtual, o comando vai depender do sistema operacional que está sendo utilizado. Caso seja o Windows, o script de ativação pode ser ativado utilizando o terminal CMD ou PowerShell, como segue.

No CMD:

```
nome_do_ambiente\Scripts\activate
```


No PowerShell:

```
nome_do_ambiente\Scripts\Activate.ps1
```

Para ativar no Linux ou no MacOS:

```
source nome_do_ambiente/bin/activate
```



IMPORTANTE

Fique atento a permissões específicas para execução do ambiente virtual que o sistema operacional pode solicitar!

É importante ressaltar que ambientes virtuais podem ser utilizados para garantir que os conjuntos de dependências de uma aplicação estejam devidamente apartados do host principal, garantindo a instalação de pacotes fora de contexto que podem aumentar o tamanho da aplicação e até mesmo gerar conflitos de pacotes no momento da execução. O uso de ambientes virtuais, associados a arquivos de requisitos (por exemplo, o requirements.txt) são práticas recomendadas que tornam o trabalho mais limpo, seguro e profissional.

O ambiente virtual pode a qualquer momento ser desativado utilizando o comando deactivate, bastando somente executar o comando, como segue:

```
deactivate
```

4 Criando seus próprios módulos

Além de consumir pacotes criados por outros desenvolvedores, o Python também permite que você crie seus próprios módulos e os torne instaláveis via pip. Essa prática é útil tanto para organizar seus projetos quanto para compartilhar bibliotecas com outras pessoas ou times.

Um pacote Python precisa seguir uma estrutura mínima para ser reconhecido corretamente. Por exemplo:

```
meu_pacote/  
  
    meu_pacote/          Diretório com o código fonte  
        __init__.py  
        modulo.py  
  
    README.md  
    setup.py  
    pyproject.toml  
    LICENSE
```

Cada arquivo dentro da estrutura de pastas possui diferente papéis, como segue:

- **__init__.py**: indica que o diretório é um pacote Python.
- **setup.py**: define metadados e instruções de instalação.
- **pyproject.toml**: especifica as dependências de build (recomendado para novas versões do Python).
- **README.md**: documentação básica.
- **LICENSE**: tipo de licença (MIT, GPL etc.).

Para versões anteriores do Python, é utilizando o arquivo setup.py para configurar a instalação, definir metadados e instalar as dependências, conforme a seguir.

```
from setuptools import setup, find_packages  
  
setup(  
    name='meu_pacote',  
    version='0.1.0',  
    description='Um pacote de exemplo para demonstração com  
pip',  
    author='Seu Nome',
```

```

author_email='seu@email.com',
packages=find_packages(),
install_requires=[], # dependências, se houver
classifiers=[
    'Programming Language :: Python :: 3',
    'License :: OSI Approved :: MIT License',
],
)

```

Explicando a composição do arquivo setup.py:

- **setuptools:** é uma biblioteca que substitui o antigo distutils e oferece ferramentas mais modernas e flexíveis para empacotar projetos Python.
- **find_packages():** detecta automaticamente todos os pacotes e subpacotes dentro de seu projeto (ou seja, diretórios com `__init__.py`). Isso evita a necessidade de listá-los manualmente.

Na função `setup()`, existem os seguintes parâmetros:

- **name:** nome do pacote como ele será reconhecido no pip e no PyPI. Evite espaços e caracteres especiais.
- **version:** versão atual de seu pacote. É uma boa prática seguir o versionamento semântico (ex: MAJOR.MINOR.PATCH).
- **description:** uma descrição curta e clara do que seu pacote faz.
- **author / author_email:** informações sobre quem criou o pacote. Isso é exibido no PyPI e ajuda a identificar o responsável.
- **packages:** lista dos diretórios que contêm código-fonte. `find_packages()` busca automaticamente por eles. Você também pode passar uma lista manual, como `['meu_pacote']`.
- **install_requires:** lista de pacotes que seu projeto depende para funcionar corretamente. Eles serão instalados automaticamente com seu pacote.

- **classifiers:** metadados padronizados utilizados pelo PyPI para classificar e descrever seu pacote. Eles ajudam na busca por pacotes com características específicas.



IMPORTANTE

Muitos projetos usam o `pyproject.toml` como padrão moderno. O `setup.py` ainda é aceito, mas pode ser substituído gradualmente.

Nos casos em que se opte pelo uso do arquivo `pyproject.toml`, segue um exemplo de preenchimento:

Exemplo 2

```
[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "meu_pacote"
version = "0.1.0"
description = "Um pacote de exemplo para demonstração com pip"
authors = [
    { name = "Seu Nome", email = "seu@email.com" }
]
license = { text = "MIT" }
readme = "README.md"
requires-python = ">=3.8"
dependencies = [
    "requests>=2.25",
    "pandas>=1.0,<2.0"
]

[project.urls]

"Homepage" = "https://github.com/seunome/meu_pacote"
"Documentação" = "https://meu-pacote.readthedocs.io"
```

Explicando a composição do arquivo `pyproject.toml`:

- `[build-system]`
- Define o backend de empacotamento e suas dependências mínimas.
- `setuptools` e `wheel` são os mais comuns.
- O `setuptools>=61.0` é necessário para suportar o novo formato via `pyproject.toml`.
- `[project]`
- Metadados do pacote: nome, versão, descrição, autor, licença etc.
- `requires-python`: especifica a versão mínima do Python.
- `dependencies`: lista de pacotes necessária para o funcionamento do projeto.
- `[project.urls]` (opcional)
- URLs úteis: site do projeto, documentação, repositório GitHub etc.

Para empacotar a solução e instalar, o próximo passo é gerar o arquivo de distribuição, através do comando:

```
python -m build
```

Para instalá-lo localmente, deve ser usado o comando `pip` buscando arquivos para a instalação local com o comando:

```
pip install .
```

O pacote também pode ser publicado no PyPI através do comando `twine`, de maneira que qualquer pessoa possa instalar seu pacote remotamente:

```
twine upload dist/*
```



PARA SABER MAIS

Publicar pacotes no PyPI (Python Package Index) é o passo final para compartilhar seu código Python com a comunidade de forma padronizada e acessível via pip install. Para isso, é necessário ter uma conta no PyPI, criar um arquivo `pyproject.toml` ou `setup.py` com os metadados do pacote, gerar os arquivos de distribuição usando a ferramenta build e, por fim, fazer o upload com o twine. Com esses passos, qualquer pessoa poderá instalar seu pacote diretamente do PyPI, o que facilita a reutilização de código, a colaboração entre desenvolvedores e a disseminação de soluções úteis de forma segura e organizada.

Considerações finais

O gerenciamento de módulos em Python representa uma das competências fundamentais para quem deseja desenvolver aplicações organizadas, eficientes e de fácil manutenção. Ao compreender o funcionamento da comunidade Python, utilizar o pip de forma adequada e adotar ambientes virtuais para isolar dependências, o desenvolvedor assegura maior controle e confiabilidade no ciclo de vida de seus projetos. Além disso, a criação e a publicação de módulos próprios ampliam as possibilidades de reutilização e colaboração, contribuindo para o fortalecimento do ecossistema Python. Dominar essas práticas não apenas facilita o desenvolvimento, como promove a profissionalização do trabalho com software livre e de código aberto.

Referências

PYCON. **[Página inicial]**. [S. l.]: PyCon, 2025. Disponível em: <https://pycon.org/>. Acesso em: 8 abr. 2025.

PYPI. **Python Software Foundation**. [S. l.]: PyPI, 2015. Disponível em: <https://www.python.org/psf-landing/>. Acesso em: 8 abr. 2025.

PYTHON BRASIL. **[Página inicial]**. [S. l.]: Python Brasil, 2015. Disponível em: <https://python.org.br/>. Acesso em: 8 abr. 2025.

PYTHON. **Python Software Foundation**. [S. l.]: Python, 2025. Disponível em: <https://www.python.org/psf-landing/>. Acesso em: 8 abr. 2025.

Sobre o autor

Gustavo Moreira Calixto possui graduação em tecnologia em informática pela Universidade Estadual de Campinas, mestrado em engenharia elétrica pela Universidade Estadual de Campinas e doutorado em ciências pela Escola Politécnica da Universidade de São Paulo, com ênfase em sistemas eletrônicos. Na Universidade de São Paulo, atuou como pesquisador do Centro Interdisciplinar de Tecnologias Interativas (CITI) na área de sistemas embarcados e sistemas multimídia, sendo responsável pela gerência do projeto Global ITV, uma colaboração entre instituições brasileiras e europeias apoiada pelo CNPq e FP7. Atuou como pesquisador visitante na The University of Texas at Austin (EUA), na Moody College of Communication, Department of Electrical and Computer Engineering. Atualmente, é docente no Centro Universitário do Senac-SP, coordenando cursos de graduação da área de TI na modalidade EaD. É professor visitante dos cursos de pós-graduação em engenharia de dados e DevOps da Universidade Presbiteriana Mackenzie. Também foi professor doutor na Fiap nos cursos de graduação e professor assistente no Insper no curso de engenharia da computação. Foi professor no Centro Paula Souza (Fatec), ministrando para os cursos de análise de sistemas e informática para negócios. Na Universidade de Guarulhos, atuou como professor doutor no curso de ciência da computação e tecnologia em gestão de TI. Tem experiência na área de tecnologia da informação e telecomunicações, voltadas a sistemas embarcados. O profissional atuou em projetos de desenvolvimento de soluções de inovação e soluções comerciais em grupos de pesquisas e consultorias da área de informática. Foi professor e coordenador adjunto dos cursos de informática da Universidade Nove de Julho e professor mestre na Faculdade Impacta de Tecnologia. Desde 2005, realiza atividades voluntárias junto ao IEEE. Desde julho de 2017 é gerente de tecnologia da Prodam/SP.

Currículo Lattes: <http://lattes.cnpq.br/6998242767220750>

